

**UNIVERSIDADE DE CAXIAS DO SUL
ÁREA DO CONHECIMENTO DE CIÊNCIAS EXATAS E
ENGENHARIAS**

MATHEUS DENDENA

**ARQUITETURAS MVC E HEXAGONAL EM UMA APLICAÇÃO
WEB: ANÁLISE COMPARATIVA DE DESEMPENHO E
MANUTENIBILIDADE**

BENTO GONÇALVES

2026

MATHEUS DENDENA

**ARQUITETURAS MVC E HEXAGONAL EM UMA APLICAÇÃO
WEB: ANÁLISE COMPARATIVA DE DESEMPENHO E
MANUTENIBILIDADE**

Trabalho de Conclusão de Curso
apresentado como requisito parcial
à obtenção do título de Cientista da
Computação na Área do Conhecimento
de Ciências Exatas e Engenharias da
Universidade de Caxias do Sul.

Orientador: Prof. Marcos Eduardo
Casa

BENTO GONÇALVES

2026

RESUMO

A definição da arquitetura de software constitui uma das decisões mais relevantes no desenvolvimento de sistemas, pois influencia diretamente atributos de qualidade como desempenho, manutenibilidade e capacidade de evolução. No contexto de aplicações web desenvolvidas com Node.js, destacam-se duas abordagens amplamente utilizadas e conceitualmente distintas: o padrão *Model-View-Controller* (MVC), reconhecido pela simplicidade estrutural e pela rapidez de desenvolvimento, e a Arquitetura Hexagonal (*Ports and Adapters*), caracterizada pelo forte desacoplamento entre a lógica de negócio e os componentes de infraestrutura. Apesar de sua ampla adoção na indústria, ainda são escassos os estudos que comparem empiricamente essas arquiteturas mantendo constantes o *runtime*, o *framework* HTTP e o banco de dados, de modo que apenas a organização interna do software seja avaliada. Este trabalho propõe um experimento controlado para comparar as arquiteturas MVC e Hexagonal sob os aspectos de desempenho e manutenibilidade, utilizando como domínio experimental um sistema de gerenciamento de tarefas organizado em quadros. As duas implementações compartilham o mesmo conjunto de tecnologias Node.js, Express e PostgreSQL diferindo exclusivamente na estrutura arquitetural adotada. A avaliação de desempenho será realizada por meio de testes de carga conduzidos com a ferramenta Autocannon, contemplando cenários de leitura, escrita e carga mista em três níveis de concorrência. Serão coletadas métricas de throughput, latência média, percentis p50, p95 e p99, além da taxa de erros. A manutenibilidade será avaliada por três eixos complementares. O primeiro consiste na análise de acoplamento estrutural com a ferramenta dependency-cruiser, utilizando as métricas Ca, Ce e Instability. O segundo aborda a testabilidade por meio da ferramenta Jest, considerando cobertura de testes por camada, tempo de execução da suíte e proporção entre mocks e fakes. O terceiro compreende experimentos controlados de substituição de componentes de infraestrutura, mensurados a partir das alterações registradas em git diff. Nesta etapa inicial da pesquisa, foram concluídos o referencial teórico, o levantamento de trabalhos relacionados, o desenho experimental e a modelagem completa da aplicação, incluindo requisitos, regras de negócio, endpoints e diagramas arquiteturais. Os resultados experimentais, a serem obtidos no TCC 2, deverão fornecer evidências empíricas que auxiliem a tomada de decisão arquitetural em projetos desenvolvidos com Node.js, Express e PostgreSQL.

Palavras-chave: Arquitetura de Software. MVC. Arquitetura Hexagonal. Desempenho. Manutenibilidade. Node.js.

ABSTRACT

Software architecture is one of the most important decisions in software development, as it directly influences quality attributes such as performance, maintainability, and system evolution. In the context of Node.js-based web applications, two widely adopted and conceptually distinct approaches stand out: the Model-View-Controller (MVC) pattern, known for its structural simplicity and rapid development, and the Hexagonal Architecture (Ports and Adapters), characterized by the strong decoupling between business logic and infrastructure components. Despite their widespread adoption in industry, there is still a lack of empirical studies comparing these architectures while keeping the runtime, HTTP framework, and database constant, so that only the internal software organization is evaluated. This work proposes a controlled experiment to compare MVC and Hexagonal architectures in terms of performance and maintainability, using a task management system organized into boards as the experimental domain. Both implementations share the same technology stack Node.js, Express, and PostgreSQL differing exclusively in their architectural structure. Performance evaluation will be conducted through load testing using the Autocannon tool, considering read-only, write-only, and mixed workload scenarios at three concurrency levels. The collected metrics will include throughput, average latency, p50, p95, and p99 latency percentiles, as well as error rate. Maintainability will be assessed through three complementary dimensions. The first consists of structural coupling analysis using the dependency-cruiser tool and the metrics Ca, Ce, and Instability. The second focuses on testability through the Jest framework, considering test coverage by layer, test suite execution time, and the proportion of mocks and fakes. The third involves controlled infrastructure component replacement experiments, measured through the changes identified in git diff. At this stage of the research, the theoretical foundation, related work review, experimental design, and complete application modeling have been concluded, including requirements, business rules, endpoints, and architectural diagrams. The experimental results, to be obtained in the second phase of the study, are expected to provide empirical evidence to support architectural decision-making in projects developed with Node.js, Express, and PostgreSQL.

Keywords: Software Architecture. MVC. Hexagonal Architecture. Performance. Maintainability. Node.js.

SUMÁRIO

1	INTRODUÇÃO	6
1.1	Problema de Pesquisa	8
1.2	Objetivos	8
1.3	Estrutura do Trabalho	9
2	FUNDAMENTAÇÃO	10
2.1	Arquitetura de Software	10
2.2	Arquitetura MVC	11
2.2.1	Componentes e Responsabilidades	11
2.2.2	Vantagens e Limitações	13
2.2.3	Aplicabilidade	14
2.3	Arquitetura Hexagonal (<i>Ports and Adapters</i>)	16
2.3.1	Componentes e Responsabilidades	16
2.3.2	Princípio da Inversão de Dependência	18
2.3.3	Relação com <i>Clean Architecture</i> e <i>Domain-Driven Design</i>	18
2.4	Análise Comparativa entre MVC e Hexagonal	19
2.5	Trabalhos Relacionados	22
3	METODOLOGIA	25
3.1	Classificação da Pesquisa	25
3.2	Ambiente Experimental	25
3.3	Ferramenta de Benchmarking	26
3.4	Cenários de Teste	26
3.4.1	Leitura Pura	27
3.4.2	Escrita Pura	27
3.4.3	Carga Mista	27
3.5	Parâmetros e Níveis de Carga	27
3.6	Métricas	28
3.6.1	Exemplo de cálculo das métricas	29
3.7	Avaliação de Manutenibilidade	31
3.7.1	Análise de Acoplamento	31
3.7.2	Análise de Testabilidade	33
3.7.3	Substituição de Componentes	34
3.8	Limitações	35
4	APLICAÇÃO EXPERIMENTAL	37

4.1	Justificativa da Escolha do Domínio	37
4.2	Requisitos Funcionais	38
4.3	Requisitos Não Funcionais	38
4.4	Modelo de Dados	39
4.4.1	Entidade User	40
4.4.2	Entidade Board	40
4.4.3	Entidade Task	41
4.5	Casos de Uso	41
4.6	Regras de Negócio	42
4.7	Endpoints da API	43
4.7.1	Recurso Users	43
4.7.2	Recurso Boards	43
4.7.3	Recurso Tasks	44
4.8	Diagrama de Arquitetura	44
4.9	Diagrama de Componentes	45
4.10	Estrutura Interna das Implementações	46
4.10.1	Implementação MVC	48
4.10.2	Implementação Hexagonal	49
4.11	Interface da Aplicação	49
5	CONSIDERAÇÕES PARCIAIS	52
	REFERÊNCIAS	54
	APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA AR- TIFICIAL	56

1 INTRODUÇÃO

O software moderno não é algo estático; ele evolui continuamente. Nesse cenário, equipes de desenvolvimento deparam-se com decisões arquiteturais que impactam diretamente o desempenho, a manutenibilidade e a escalabilidade das aplicações. Essas decisões, muitas vezes tomadas ainda nas fases iniciais do projeto, não apenas orientam a construção do sistema, mas também determinam sua capacidade de adaptação diante de novas demandas, correções de bugs e integrações futuras. Quando essa base estrutural é frágil, até mesmo pequenas alterações podem se transformar em obstáculos significativos, gerando retrabalho, aumento da complexidade e dificuldades de manutenção.

Em sistemas pouco estruturados, mudanças simples tendem a se tornar problemáticas. A ausência de uma arquitetura bem definida frequentemente leva ao chamado “código espaguete” (HOMER, 2009; BELLACOSA, 2021), no qual responsabilidades se misturam em diferentes partes do sistema. Como consequência, o código se torna progressivamente mais difícil de compreender, testar e evoluir. Nesse contexto, a engenharia de software propõe padrões e modelos arquiteturais que auxiliam na organização do código e na mitigação desses problemas.

Ao longo das últimas décadas, diversos padrões arquiteturais se consolidaram tanto na academia quanto na indústria. Um dos mais influentes é o *Model-View-Controller* (MVC), proposto por Trygve Reenskaug entre 1978 e 1979. Esse padrão organiza a aplicação em três camadas distintas: o *Model*, responsável pelos dados e regras de negócio; a *View*, encarregada da apresentação; e o *Controller*, que atua como intermediário entre as duas. Essa separação contribui para uma melhor organização do sistema e foi amplamente adotada no desenvolvimento de aplicações web (REENSKAUG, 1979).

Outro modelo relevante é a Arquitetura Hexagonal, também conhecida como *Ports and Adapters*, proposta por Alistair Cockburn em 2005. Diferentemente do MVC, essa abordagem enfatiza um isolamento mais rigoroso da lógica de negócio em relação a elementos externos, como bancos de dados, APIs e interfaces de usuário. No núcleo da aplicação o domínio permanece independente de tecnologias específicas, comunicando-se com o exterior por meio de portas e adaptadores bem definidos (COCKBURN, 2005). Essa característica favorece a testabilidade e facilita a substituição de componentes de infraestrutura sem impacto direto na lógica central do sistema.

Apesar de suas vantagens, a Arquitetura Hexagonal é frequentemente alvo de discussões na literatura e em comunidades técnicas, especialmente quanto ao risco de *overengineering*. No entanto, conforme argumenta (MIGUEL, 2025), o problema raramente está na arquitetura em si, mas na forma como ela é aplicada. Camadas mal definidas, contratos inconsistentes e a falta de compreensão dos princípios fundamentais são fatores que contribuem para a complexidade

desnecessária. Além disso, questões organizacionais também influenciam: equipes pressionadas por prazos tendem a negligenciar boas práticas, o que pode transformar uma arquitetura pensada para organizar o sistema em uma fonte adicional de dificuldade. Assim, a Arquitetura Hexagonal tende a se tornar excessiva apenas quando aplicada sem considerar o contexto do sistema, como em aplicações simples, protótipos ou soluções sem previsão de manutenção e expansão, ou quando implementada por equipes sem o preparo adequado.

Casos reais ajudam a ilustrar esses conceitos. Em um artigo do Netflix Technology Blog, a equipe de Studio Workflow relatou que a adoção da Arquitetura Hexagonal permitiu isolar completamente a lógica de negócio das fontes de dados externas (DAMIR; SERGII, 2020). Como resultado, foi possível migrar o consumo de dados de uma API JSON para GraphQL em apenas duas horas, sem qualquer modificação no domínio da aplicação. Esse exemplo demonstra, na prática, como o uso de *Ports and Adapters* possibilita mudanças de infraestrutura sem impactar o núcleo do sistema.

Já o GitHub oferece um exemplo consolidado do uso do padrão MVC em larga escala. A plataforma foi originalmente construída com Ruby on Rails e mantém essa base até hoje. Segundo seu blog de engenharia, trata-se de um monólito com quase dois milhões de linhas de código, desenvolvido por mais de mil engenheiros e com deploys realizados diversas vezes ao dia (UCHITELLE; THOMAS, 2023). Nesse contexto, o MVC contribuiu para o crescimento acelerado da aplicação e para a integração de novos desenvolvedores ao projeto.

Por outro lado, nem todos os cenários favorecem o MVC. Durante a conferência F8 de 2014, representantes do Facebook afirmaram que o padrão não atendia mais às necessidades da empresa, especialmente no *frontend*. Conforme relatado pela InfoQ, o fluxo bidirecional de dados entre *models* e *views* tornava o comportamento da aplicação difícil de prever e depurar, já que alterações em um *model* podiam gerar efeitos colaterais em múltiplas *views* (INFOQ, 2014). Como resposta, foi adotado o padrão Flux, baseado em fluxo de dados unidirecional, que posteriormente deu origem ao React.

É importante destacar que esse problema manifesta-se de forma mais intensa no *frontend*, onde múltiplas *views* compartilham estado. Ainda assim, ele evidencia uma limitação conceitual do MVC: a dificuldade em rastrear os impactos de mudanças nos modelos. No *backend*, essa limitação pode aparecer de outras formas, como no acoplamento implícito entre regras de negócio e infraestrutura ou no crescimento descontrolado de *controllers*.

Por fim, a empresa Integral.io documentou os desafios enfrentados ao adotar a Arquitetura Hexagonal em um sistema de crédito. Embora a migração tenha sido concluída com sucesso, a equipe relatou aumento na complexidade do projeto e uma curva de aprendizado significativa, especialmente para desenvolvedores não familiarizados com o padrão (INTEGRAL.IO, 2024). Isso exigiu investimentos em capacitação e acompanhamento contínuo para garantir a correta aplicação dos princípios arquiteturais. O caso evidencia que, mesmo abordagens bem fundamentadas, quando aplicadas sem o devido preparo, podem resultar em retrabalho, atrasos

e frustração.

1.1 PROBLEMA DE PESQUISA

A definição da arquitetura de software representa uma das decisões mais relevantes nas fases iniciais de um projeto, pois influencia diretamente aspectos como desempenho, manutenibilidade e evolução do sistema. No entanto, essa escolha frequentemente é realizada sem o apoio de dados empíricos que evidenciem seus impactos práticos, ou ainda sem considerar adequadamente as particularidades do contexto em que a aplicação será inserida.

Diante desse cenário, torna-se pertinente investigar de forma mais aprofundada os efeitos das diferentes abordagens arquiteturais, especialmente no que diz respeito ao seu comportamento em aplicações reais.

Assim, estabelece-se as seguintes questões de pesquisa:

Quais diferenças de desempenho e manutenibilidade podem ser identificadas entre as arquiteturas MVC e Hexagonal no desenvolvimento de uma aplicação web? É possível indicar a abordagem mais adequada a partir dos dados coletados?

1.2 OBJETIVOS

Este trabalho tem como objetivo comparar as arquiteturas MVC e Hexagonal no contexto do desenvolvimento de uma aplicação web, analisando tanto o desempenho sob carga quanto a manutenibilidade, com o propósito de identificar possíveis cenários em que cada abordagem apresenta melhores características considerando os critérios analisados.

Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

1. Caracterizar as arquiteturas MVC e Hexagonal, apresentando seus princípios, estruturas e formas de aplicação no desenvolvimento de sistemas web.
2. Desenvolver uma aplicação web equivalente utilizando ambas as arquiteturas, assegurando a paridade funcional para viabilizar uma comparação justa.
3. Levantar e analisar trabalhos relacionados, buscando referências que contribuam para a definição e fundamentação da metodologia adotada.
4. Mensurar e comparar métricas de desempenho entre as arquiteturas, como requisições por segundo, latência e taxa de transferência, utilizando a ferramenta Autocannon.
5. Avaliar a manutenibilidade de cada abordagem com base em critérios como testabilidade, grau de acoplamento entre componentes, separação de responsabilidades e facilidade de substituição de componentes.

6. Analisar os resultados obtidos, identificando os contextos em que cada arquitetura pode ser considerada uma melhor escolha.

1.3 ESTRUTURA DO TRABALHO

O presente trabalho está organizado em seis capítulos, descritos a seguir:

- Capítulo 1 - Introdução: Apresenta a contextualização do tema, o problema de pesquisa, os objetivos geral e específicos.
- Capítulo 2 - Fundamentação: Conceitos teóricos que embasam o trabalho. São caracterizadas as arquiteturas MVC e Hexagonal, com descrição de seus componentes, responsabilidades, vantagens e limitações. O capítulo inclui ainda uma comparação direta entre os dois padrões e uma revisão dos trabalhos relacionados que adotam metodologias similares à proposta.
- Capítulo 3 - Metodologia: Descreve os procedimentos metodológicos adotados, incluindo a classificação da pesquisa, o ambiente experimental, a ferramenta de benchmarking utilizada, os cenários e níveis de carga definidos, as métricas de desempenho coletadas e os critérios de avaliação de manutenibilidade, abrangendo análise de acoplamento, testabilidade e substituição de componentes.
- Capítulo 4 – Aplicação Experimental: apresenta a modelagem e implementação da aplicação utilizada nos experimentos, incluindo requisitos funcionais e não funcionais, modelo de dados, regras de negócio, endpoints, diagramas arquiteturais e estrutura interna das implementações MVC e Hexagonal.
- Capítulo 5 – Considerações Finais: apresenta as conclusões parciais do trabalho, destacando os principais aspectos discutidos, as limitações identificadas e os encaminhamentos previstos para a continuidade da pesquisa no TCC 2 juntamente com o planejamento das atividades previstas para a etapa seguinte, contemplando o desenvolvimento das duas implementações, a execução dos experimentos de desempenho e de manutenibilidade, a coleta e análise dos resultados e a redação final do trabalho.

2 FUNDAMENTAÇÃO

2.1 ARQUITETURA DE SOFTWARE

A arquitetura de software constitui um dos pilares da engenharia de software moderna, pois envolve as decisões estruturais que definem a organização e o comportamento de um sistema. Bass, Clements e Kazman (2012), em uma das principais obras da área, definem arquitetura de software como a estrutura ou conjunto de estruturas de um sistema, englobando seus elementos, as propriedades externamente visíveis desses elementos e as relações entre eles (BASS; CLEMENTS; KAZMAN, 2012). Essa definição evidencia que a arquitetura vai além de diagramas ou da organização de diretórios, representando, na prática, as decisões fundamentais que orientam tanto a construção quanto a comunicação entre os componentes de um sistema.

Quando não há uma arquitetura bem definida, é comum que o sistema evolua de forma desorganizada, com responsabilidades mal distribuídas entre seus componentes. Esse cenário frequentemente leva ao chamado “código espaguete” (HOMER, 2009; BELLACOSA, 2021), no qual as dependências se tornam confusas e difíceis de rastrear. Como consequência, o código tende a se tornar progressivamente mais complexo, dificultando sua compreensão, manutenção e evolução.

Na prática, os impactos de uma arquitetura inadequada se manifestam de diversas formas: aumento no tempo de desenvolvimento de novas funcionalidades, maior propensão à introdução de erros em alterações simples, dificuldades de integração com sistemas externos e resistência à adoção de novas tecnologias. Esses problemas evidenciam a importância de decisões arquiteturais bem fundamentadas desde o início do projeto.

Conceitos essenciais na análise arquitetural são os atributos de qualidade de software. Sommerville (2011) destaca que, além de atender aos requisitos funcionais, os sistemas devem satisfazer requisitos não funcionais, como desempenho, manutenibilidade, testabilidade e escalabilidade (SOMMERVILLE, 2011). Esses atributos, frequentemente denominados *quality attributes*, são diretamente influenciados pelas decisões arquiteturais adotadas. Segundo Bass, Clements e Kazman (2012), a arquitetura de software exerce papel fundamental na determinação dos atributos de qualidade, pois define as principais estruturas do sistema e as relações entre seus componentes. Dessa forma, decisões arquiteturais representam escolhas de compromisso (*trade-offs*) entre diferentes características, uma vez que otimizar determinado atributo pode impactar outros (BASS; CLEMENTS; KAZMAN, 2012).

Grande parte dos padrões arquiteturais modernos se apoia em dois princípios fundamentais: coesão e acoplamento. A coesão refere-se ao grau em que os elementos de um módulo estão relacionados entre si, contribuindo para uma única responsabilidade bem definida. Já o

acoplamento diz respeito ao nível de dependência entre diferentes módulos. De forma geral, a literatura aponta que sistemas bem projetados devem buscar alta coesão e baixo acoplamento, princípio que fundamenta tanto o MVC quanto a Arquitetura Hexagonal, embora cada um o aplique de maneira distinta (BASS; CLEMENTS; KAZMAN, 2012; SOMMERVILLE, 2011).

Outro conceito central é o da Separação de Responsabilidades (*Separation of Concerns* – SoC), proposto por (DIJKSTRA, 1974), que estabelece que diferentes aspectos de um sistema devem ser tratados de forma independente. Esse princípio busca evitar que uma preocupação interfira diretamente em outra, promovendo maior organização e clareza estrutural. Tanto o MVC quanto a Arquitetura Hexagonal compartilham esse fundamento teórico, ainda que o implementem de formas diferentes.

Neste trabalho, a análise concentra-se nas arquiteturas MVC e Hexagonal. Essa escolha se justifica por dois fatores principais. Em primeiro lugar, ambas são amplamente utilizadas no desenvolvimento de aplicações web, contando com vasta adoção tanto na academia quanto na indústria. Em segundo lugar, representam abordagens conceitualmente distintas: enquanto o MVC privilegia a simplicidade, a produtividade e a rapidez de desenvolvimento, a Arquitetura Hexagonal enfatiza o isolamento do domínio e a substituíbilidade dos componentes de infraestrutura. Essa diferença de enfoque torna a comparação entre essas arquiteturas especialmente relevante para apoiar decisões em diferentes contextos de projeto.

2.2 ARQUITETURA MVC

O padrão arquitetural *Model-View-Controller* (MVC) foi concebido por Trygve Reenskaug em 1979, durante seu trabalho no Xerox PARC, no contexto da linguagem Smalltalk. Sua motivação inicial estava relacionada a um problema prático: como estruturar aplicações interativas de forma que a interface com o usuário pudesse evoluir independentemente da lógica de negócio? A solução proposta foi a divisão da aplicação em três componentes com responsabilidades distintas e complementares (REENSKAUG, 1979).

Desde então, o MVC se consolidou como um dos padrões arquiteturais mais influentes da engenharia de software. Ao longo das décadas, foi amplamente adotado em diferentes linguagens e plataformas, estando presente tanto em *frameworks* desktop, como o Cocoa (Apple), quanto em *frameworks web*, como Ruby on Rails, Django, Laravel e Spring MVC. Essa ampla adoção contribuiu para sua consolidação como uma abordagem padrão no desenvolvimento de aplicações, especialmente no contexto web, onde a separação entre apresentação e lógica de negócio se mostra essencial.

2.2.1 Componentes e Responsabilidades

A arquitetura *Model-View-Controller* (MVC), proposta por Reenskaug (1979), organiza a aplicação em três componentes principais, cada um responsável por um aspecto distinto da

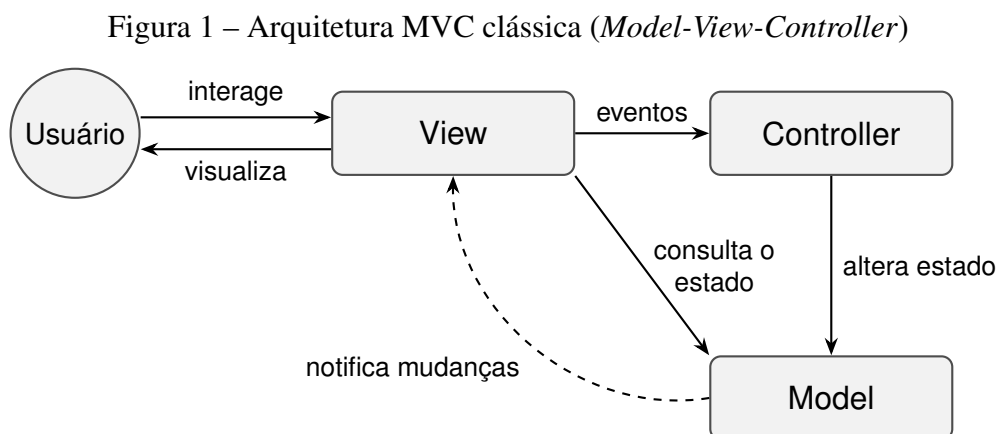
interação entre o usuário e o sistema.

Model (Modelo): representa o estado da aplicação e encapsula as regras de negócio. O *Model* é responsável por manter e manipular os dados da aplicação, sendo independente da interface com o usuário. No MVC original, o *Model* não possui conhecimento sobre como seus dados serão apresentados nem sobre a forma como as entradas do usuário são realizadas.

View (Visão): corresponde à representação visual do estado do *Model* para o usuário. A *View* consulta o *Model* para obter as informações necessárias à apresentação e pode ser notificada sempre que seu estado é alterado, permitindo a atualização automática da interface por meio do padrão Observer. Além de exibir informações, a *View* também recebe as interações do usuário como cliques e entradas de dados e as repassa ao *Controller* para processamento. Sua responsabilidade restringe-se, portanto, à camada de apresentação e captura de eventos, sem incorporar regras de negócio.

Controller (Controlador): é responsável por interpretar os eventos recebidos da *View* e convertê-los em operações sobre o *Model*. O *Controller* atua como mediador entre a camada de apresentação e o domínio da aplicação, definindo quais operações devem ser executadas em resposta às interações do usuário.

A Figura 1 apresenta a organização conceitual do MVC proposto por Reenskaug (1979), evidenciando a separação entre *Model*, *View* e *Controller* e o fluxo de interação entre esses componentes.



Fonte: Adaptado de Lemos *et al.* (2013).

Observa-se, na Figura 1, que o usuário interage diretamente com a *View*, seja recebendo informações apresentadas pela interface, seja enviando entradas que são capturadas e repassadas ao *Controller*. Este, por sua vez, interpreta os eventos recebidos e executa as operações correspondentes sobre o *Model*. Sempre que o estado do *Model* é alterado, a *View* é notificada automaticamente por meio do padrão Observer, atualizando a interface sem que o *Controller* precise coordenar essa atualização. Essa organização evidencia a separação de responsabilidades proposta por Reenskaug (1979), na qual o *Model* permanece independente da interface e

do mecanismo de entrada, permitindo que diferentes *Views* representem um mesmo estado da aplicação.

No contexto de aplicações web, o fluxo de processamento ocorre, em geral, da seguinte forma: o usuário envia uma requisição HTTP ao servidor, que é interceptada pelo Controller. Esse componente é responsável por validar os parâmetros recebidos, acionar os serviços ou componentes responsáveis pela manipulação dos dados no *Model* e, posteriormente, selecionar a *View* apropriada, fornecendo as informações necessárias para sua renderização. Por fim, a resposta é retornada ao usuário no formato esperado, como HTML ou JSON.

Embora esse fluxo seja predominante em aplicações web, o padrão MVC pode assumir diferentes implementações conforme o contexto. Em aplicações desktop, por exemplo, é comum utilizar mecanismos baseados no padrão Observer para estabelecer uma comunicação reativa entre *Model* e *View*, permitindo que alterações no estado dos dados sejam propagadas automaticamente para a interface. Além disso, padrões complementares como Data Access Object (DAO) ou Repository podem ser empregados para separar a persistência dos dados da lógica do *Model*, reduzindo o acoplamento com mecanismos específicos de armazenamento.

2.2.2 Vantagens e Limitações

Entre as principais vantagens da arquitetura MVC no desenvolvimento de aplicações web, destacam-se:

- **Separação de responsabilidades:** a divisão em camadas distintas contribui para uma organização mais clara do sistema, reduzindo o acoplamento entre a lógica de negócio e a apresentação, além de facilitar manutenção e evolução do código (SOMMERVILLE, 2011; BASS; CLEMENTS; KAZMAN, 2012).
- **Velocidade de desenvolvimento:** frameworks baseados em MVC, como Ruby on Rails e Laravel, oferecem convenções e ferramentas que aceleram o desenvolvimento, tornando essa abordagem especialmente adequada para projetos com prazos reduzidos.
- **Ampla adoção e ecossistema maduro:** a popularidade do MVC resulta em vasta documentação, comunidades ativas e um grande número de bibliotecas e ferramentas de suporte, facilitando sua adoção e uso contínuo.
- **Facilidade de onboarding:** por ser amplamente difundido e possuir uma estrutura bem conhecida, o MVC reduz a curva de aprendizado para novos desenvolvedores, facilitando sua integração em projetos existentes.

Por outro lado, o padrão MVC apresenta limitações que se tornam mais evidentes em aplicações de maior porte ou com regras de negócio mais complexas:

- **Acoplamento implícito à infraestrutura:** em algumas implementações do MVC, o *Model* concentra responsabilidades relacionadas ao acesso e persistência dos dados, estabelecendo uma dependência direta entre a lógica da aplicação e tecnologias específicas de armazenamento. Esse acoplamento pode dificultar a substituição de componentes de infraestrutura e aumentar o impacto de alterações no sistema. Entretanto, esse problema pode ser reduzido por meio da adoção de padrões complementares, como *Repository* ou *Data Access Object* (DAO), que isolam as operações de persistência e promovem uma separação mais clara entre as regras da aplicação e os mecanismos de armazenamento.
- **Crescimento desordenado do Controller:** na ausência de disciplina arquitetural, os *Controllers* tendem a concentrar responsabilidades indevidas como validações complexas, orquestração de serviços e transformação de dados resultando nos chamados *fat controllers*, que comprometem a manutenibilidade do sistema.
- **Propagação de efeitos colaterais entre camadas:** alterações no *Model* podem impactar *Controllers*, serializadores e *Views* de forma pouco previsível, especialmente quando há forte acoplamento entre essas camadas. Esse tipo de problema foi evidenciado no contexto de MVC aplicado ao frontend do Facebook, onde o fluxo bidirecional entre *Model* e *View* tornou o sistema difícil de prever e depurar, motivando a adoção do padrão Flux (INFOQ, 2014). Embora aplicações *backend*, tipicamente *stateless*, não reproduzam exatamente esse comportamento, o princípio permanece: quanto maior o acoplamento entre camadas, maior a dificuldade de rastrear e isolar mudanças.

2.2.3 Aplicabilidade

A arquitetura MVC mostra-se especialmente adequada para aplicações do tipo CRUD (*Create, Read, Update, Delete*), portais web, sistemas de gerenciamento de conteúdo e APIs REST com lógica de negócio de complexidade moderada. Também é frequentemente adotada em projetos conduzidos por equipes pequenas e médias, nas quais a velocidade de desenvolvimento e a organização inicial do código são fatores prioritários.

Entretanto, a adoção do MVC não limita necessariamente a capacidade de evolução de sistemas de grande escala. Um exemplo relevante é o GitHub, inicialmente desenvolvido utilizando Ruby on Rails, que ao longo de sua evolução adotou práticas de modularização para organizar um sistema de grande porte mantido por uma grande equipe de engenharia e com alta frequência de entregas (UCHITELLE; THOMAS, 2023). Esse caso evidencia que aplicações baseadas em MVC podem evoluir para arquiteturas mais estruturadas quando combinadas com estratégias como separação de módulos, definição de limites internos e boas práticas de engenharia.

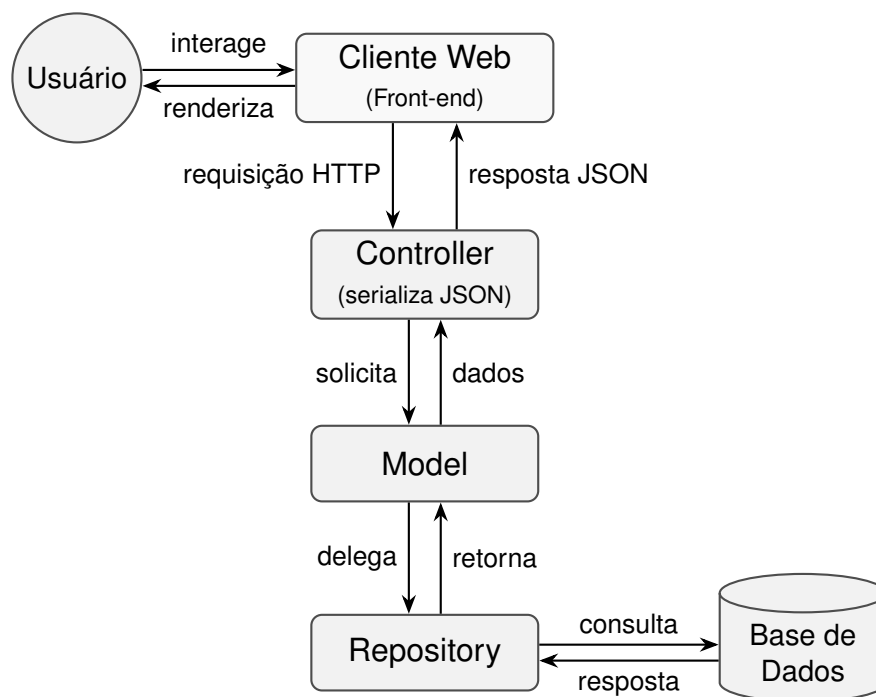
Apesar dessas possibilidades, algumas limitações do MVC tornam-se relevantes no contexto deste estudo. Em implementações sem uma organização adequada, a concentração de res-

ponsabilidades nos *Controllers*, o acoplamento entre componentes e a proximidade entre regras de negócio e infraestrutura podem impactar a manutenibilidade do sistema. Além disso, essas decisões estruturais podem influenciar o comportamento da aplicação sob carga.

É importante destacar que o MVC original de Reenskaug (1979) foi concebido para aplicações desktop, nas quais a View observa diretamente o *Model* por meio do padrão Observer. No contexto de APIs REST, essa estrutura é adaptada: a View tradicional responsável pela renderização de páginas HTML é substituída pela serialização da resposta em JSON realizada pelo *Controller*. Adicionalmente, o acesso ao banco de dados é delegado a uma camada de *Repository* separada do *Model*, prática consolidada no desenvolvimento de APIs com Node.js e Express. Neste trabalho, adota-se a variante MVC para APIs REST denominação utilizada para descrever a adaptação do padrão original ao contexto de APIs sem camada de apresentação HTML, conforme detalhado na Seção 4.10.

A Figura 2 ilustra o fluxo nessa variante, evidenciando a separação entre *Controller*, *Model* e a camada de *Repository* responsável pelo acesso ao banco de dados

Figura 2 – Arquitetura MVC adotada neste trabalho (API REST)



Fonte: O Autor (2026).

Neste trabalho, o termo MVC refere-se à adaptação do padrão *Model-View-Controller* para APIs REST implementadas em Node.js e Express. Nessa variante, o Controller atua como ponto de entrada das requisições HTTP, o Model concentra as regras de negócio, a persistência é delegada a uma camada de *Repository* e a View tradicional é substituída pela serialização das respostas em JSON. Essa organização representa uma adaptação amplamente utilizada em aplicações backend contemporâneas.

Diante disso, torna-se pertinente investigar empiricamente em que medida essas características afetam métricas de desempenho, como latência e throughput, bem como aspectos relacionados à manutenibilidade. Essa análise contribui para compreender os limites práticos do MVC e compará-los com abordagens arquiteturais alternativas, como a Arquitetura Hexagonal.

2.3 ARQUITETURA HEXAGONAL (*PORTS AND ADAPTERS*)

A Arquitetura Hexagonal, também conhecida como *Ports and Adapters*, foi proposta por Alistair Cockburn em 2005. Diferentemente do MVC, que organiza o sistema em camadas horizontais, essa abordagem adota um modelo estrutural concêntrico, no qual um núcleo central correspondente ao domínio da aplicação permanece isolado e se comunica com o ambiente externo exclusivamente por meio de interfaces bem definidas, denominadas portas (*ports*) e adaptadores (*adapters*) (COCKBURN, 2005).

A motivação central dessa proposta está na necessidade de resolver um problema recorrente no desenvolvimento de software: a forte dependência da lógica de negócio em relação a detalhes de implementação, como bancos de dados, *frameworks web* e bibliotecas externas. Esse acoplamento tende a dificultar a realização de testes, além de tornar a evolução do sistema mais custosa e arriscada. Para mitigar esse problema, a Arquitetura Hexagonal propõe a inversão dessa dependência, estabelecendo que o domínio da aplicação deve ser independente da infraestrutura. Nesse modelo, não é o domínio que depende de tecnologias externas, mas sim a infraestrutura que se adapta às regras e interfaces definidas pelo domínio.

2.3.1 Componentes e Responsabilidades

A Arquitetura Hexagonal organiza a aplicação em três zonas conceituais distintas, cada uma com responsabilidades bem definidas:

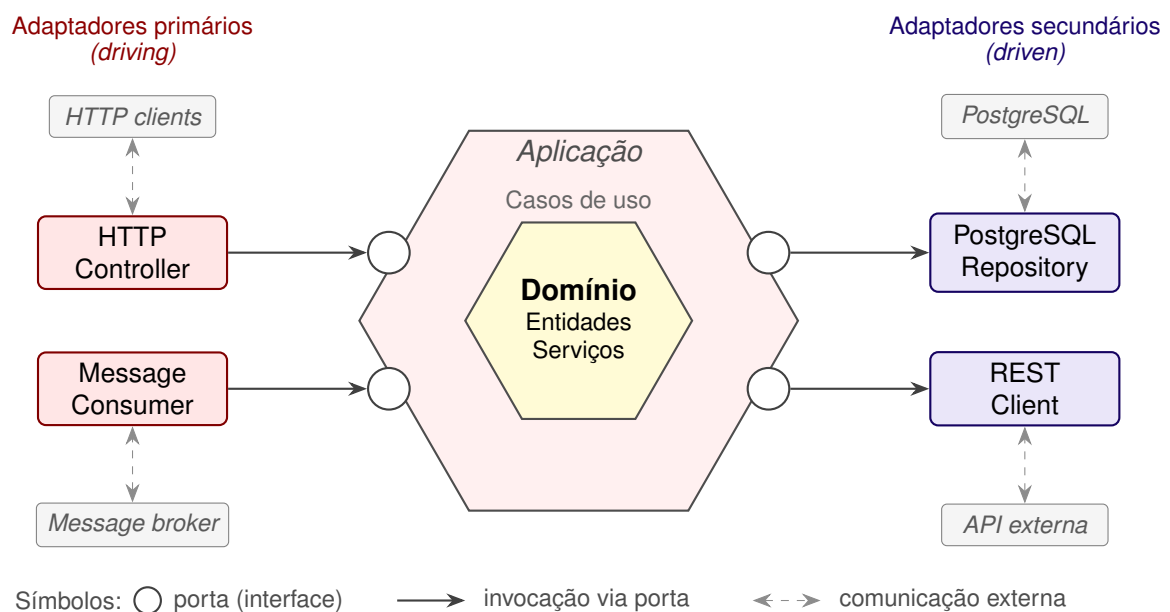
Núcleo do Domínio (*Domain / Application Core*): corresponde ao centro da arquitetura, onde estão concentradas as regras de negócio da aplicação. Essa camada inclui entidades, objetos de valor, serviços de domínio e casos de uso. O núcleo é completamente independente de tecnologias externas, não possuindo conhecimento sobre *frameworks*, bancos de dados, protocolos HTTP ou quaisquer outros detalhes de infraestrutura. Essa independência favorece a testabilidade e contribui para a longevidade do sistema.

Portas (*Ports*): representam os contratos de comunicação definidos pelo núcleo da aplicação com o ambiente externo. Essas portas são geralmente expressas por meio de interfaces ou outras formas de abstração e delimitam como o sistema recebe informações ou solicita recursos externos. As portas podem ser classificadas em dois grupos: portas primárias (*inbound*), que definem os pontos de entrada da aplicação, permitindo que agentes externos acionem casos de uso; e portas secundárias (*outbound*), que representam as necessidades do núcleo em relação a recursos externos, como persistência de dados, serviços de terceiros ou sistemas de mensageria.

Adaptadores (Adapters): são componentes externos ao núcleo responsáveis por implementar ou utilizar as portas definidas pela aplicação. Os adaptadores de entrada (*inbound adapters*) traduzem requisições externas, como chamadas HTTP, comandos ou eventos, para operações compreendidas pelo domínio. Já os adaptadores de saída (*outbound adapters*) implementam os mecanismos necessários para comunicação com recursos externos, como bancos de dados, APIs externas ou sistemas de armazenamento. Dessa forma, os detalhes tecnológicos permanecem isolados, permitindo sua substituição sem modificar o núcleo da aplicação.

A Figura 3 ilustra a estrutura da Arquitetura Hexagonal, destacando a separação entre o núcleo do domínio, as portas e os adaptadores, bem como as relações de comunicação estabelecidas entre esses elementos.

Figura 3 – Arquitetura Hexagonal (*Ports and Adapters*)



Fonte: Adaptado de Cockburn (2005).

A Figura 3 evidencia que todas as interações com o sistema ocorrem por meio de portas, reforçando o desacoplamento entre o núcleo do domínio e os elementos externos. Esse isolamento é um dos principais fatores que contribuem para a elevada testabilidade proporcionada por essa arquitetura.

A denominação “hexagonal” não deve ser interpretada como uma referência direta à quantidade de elementos ou camadas da arquitetura. O formato hexagonal foi utilizado por Cockburn como uma representação visual para destacar a ideia de que uma aplicação pode possuir múltiplos pontos de interação com o ambiente externo, sem que exista uma hierarquia definida entre essas interfaces (COCKBURN, 2005). Dessa forma, os lados da figura não representam componentes específicos do sistema, mas sim uma abstração das possíveis portas de entrada e saída pelas quais a aplicação pode se comunicar.

Essa representação auxilia na compreensão de que o núcleo da aplicação deve perma-

necer independente dos mecanismos externos. Uma aplicação pode receber estímulos por diferentes meios, como requisições HTTP, interfaces de linha de comando ou eventos, e também se comunicar com recursos externos como bancos de dados, serviços de terceiros e sistemas de mensageria. Esses elementos são tratados como adaptadores conectados ao núcleo por meio de portas bem definidas, permitindo que as tecnologias externas sejam substituídas sem modificar as regras centrais do domínio.

Por outro lado, a Arquitetura Hexagonal introduz um nível adicional de abstração que, embora favoreça a modularidade e a testabilidade, pode resultar em maior complexidade estrutural. Esse aspecto levanta uma questão relevante no contexto deste trabalho: até que ponto o isolamento do domínio compensa possíveis impactos em desempenho? A resposta a essa questão exige uma análise baseada em evidências empíricas, que será desenvolvida nos capítulos seguintes.

2.3.2 Princípio da Inversão de Dependência

Um dos conceitos fundamentais para a compreensão da Arquitetura Hexagonal é o Princípio da Inversão de Dependência (*Dependency Inversion Principle* - DIP), integrante dos princípios SOLID propostos por Robert C. Martin. Esse princípio estabelece que módulos de alto nível responsáveis pelas regras de negócio não devem depender de módulos de baixo nível relacionados à infraestrutura, mas que ambos devem depender de abstrações (MARTIN, 2017; PALERMO, 2008). No contexto da Arquitetura Hexagonal, essas abstrações são representadas pelas portas.

Na prática, isso significa que o domínio da aplicação define as interfaces de que necessita, enquanto os componentes de infraestrutura são responsáveis por implementá-las. Dessa forma, a dependência é invertida: não é o domínio que se adapta à infraestrutura, mas a infraestrutura que se ajusta às necessidades do domínio.

Essa inversão é o mecanismo que viabiliza importantes benefícios da arquitetura. Entre eles, destaca-se a possibilidade de executar testes de unidade do domínio de forma isolada, sem a necessidade de dependências externas, utilizando implementações em memória (*in-memory*) das portas secundárias. Além disso, esse princípio permite maior flexibilidade na substituição de tecnologias.

2.3.3 Relação com *Clean Architecture* e *Domain-Driven Design*

A Arquitetura Hexagonal apresenta forte relação com outras abordagens contemporâneas de organização de software, como a *Clean Architecture* e o *Domain-Driven Design* (DDD). Essas abordagens compartilham um princípio fundamental: a priorização do domínio da aplicação e a redução do acoplamento entre as regras de negócio e os detalhes externos do sistema.

A *Clean Architecture*, proposta por Robert C. Martin, organiza a aplicação em camadas

concêntricas, definindo que as dependências devem sempre apontar em direção ao núcleo do sistema. Dessa forma, regras de negócio permanecem isoladas de elementos externos, como frameworks, bancos de dados e interfaces de usuário. Essa organização reforça o Princípio da Inversão de Dependência (DIP), no qual componentes de alto nível não devem depender diretamente de componentes de baixo nível, mas sim de abstrações (MARTIN, 2017).

De maneira semelhante, a Arquitetura Hexagonal busca proteger o núcleo da aplicação por meio da separação entre o domínio e os mecanismos externos de comunicação. Entretanto, sua principal contribuição está na definição de limites claros entre a aplicação e o ambiente externo, utilizando portas e adaptadores para controlar as interações com elementos como APIs, bancos de dados e serviços de terceiros (COCKBURN, 2005).

O *Domain-Driven Design* (DDD), proposto por Eric Evans, complementa essas abordagens ao concentrar-se na construção de modelos de domínio alinhados ao problema de negócio. Por meio de conceitos como entidades, objetos de valor, agregados e serviços de domínio, o DDD busca tornar as regras da aplicação mais expressivas e próximas da linguagem utilizada pelos especialistas do domínio (EVANS, 2003).

Apesar de não serem equivalentes, essas abordagens podem ser aplicadas em conjunto. O DDD contribui para a modelagem das regras e conceitos do domínio, enquanto a Arquitetura Hexagonal e a *Clean Architecture* auxiliam na organização das dependências e no isolamento da infraestrutura. Essa combinação favorece sistemas mais flexíveis, nos quais mudanças em tecnologias externas tendem a causar menor impacto no núcleo da aplicação.

Além disso, essa separação influencia diretamente a manutenibilidade e a testabilidade do software. Com o domínio desacoplado de recursos externos, torna-se possível validar regras de negócio por meio de testes isolados, sem a necessidade de inicializar componentes como servidores HTTP ou bancos de dados. Assim, a arquitetura contribui para uma evolução mais segura do sistema ao longo do seu ciclo de vida (MARTIN, 2017; EVANS, 2003).

É importante destacar que implementações contemporâneas da Arquitetura Hexagonal frequentemente combinam os princípios originais de Cockburn (2005) com conceitos de *Clean Architecture* (MARTIN, 2017) e *Domain-Driven Design* (EVANS, 2003), subdividindo o núcleo em camadas de domínio e aplicação. Essa é a abordagem adotada neste trabalho, conforme detalhado na Seção 4.10.

2.4 ANÁLISE COMPARATIVA ENTRE MVC E HEXAGONAL

As arquiteturas MVC e Hexagonal partem de um mesmo princípio fundamental a Separação de Responsabilidades (*Separation of Concerns* – SoC), proposta por (DIJKSTRA, 1974), porém seguem abordagens distintas em sua aplicação prática, destacando-se em diferentes contextos. O MVC prioriza a simplicidade e a produtividade, organizando a aplicação em camadas diretas e de fácil compreensão (REENSKAUG, 1979). Em contrapartida, a Arquitetura Hexago-

nal adota uma abordagem mais rigorosa quanto ao isolamento da lógica de negócio, utilizando contratos explícitos as chamadas portas e adaptadores para mediar a comunicação com o ambiente externo (COCKBURN, 2005). Esse nível adicional de abstração, embora traga benefícios estruturais, implica maior complexidade inicial.

No que se refere ao acoplamento, o MVC frequentemente admite dependências implícitas entre a lógica de negócio e a infraestrutura. Um exemplo recorrente é o acesso direto ao banco de dados a partir do *Model*, prática discutida tanto na literatura acadêmica (BASS; CLEMENTS; KAZMAN, 2012) quanto em casos reais, como a evolução da arquitetura do Facebook no *frontend*, que evidenciou limitações do modelo tradicional (INFOQ, 2014). Por outro lado, a Arquitetura Hexagonal busca eliminar esse tipo de dependência, exigindo que toda interação com sistemas externos seja realizada por meio de interfaces bem definidas. Essa abordagem reduz o acoplamento e está alinhada ao Princípio da Inversão de Dependência (MARTIN, 2017).

Em termos de testabilidade, essa diferença torna-se ainda mais evidente. A Arquitetura Hexagonal, ao manter o domínio independente da infraestrutura, facilita a escrita de testes unitários isolados, dispensando o uso de recursos externos por meio de implementações em memória das portas (COCKBURN, 2005). No MVC, essa separação nem sempre é rigorosamente aplicada, o que frequentemente demanda o uso intensivo de *mocks* e pode tornar os testes mais sensíveis a mudanças internas (MARTIN, 2017).

Quanto ao desempenho, existe a possibilidade de que o maior número de abstrações na Arquitetura Hexagonal introduza um custo adicional. Estudos, como o de Pereira e Diniz (2022), sugerem que arquiteturas mais simples, com menor número de camadas, podem apresentar vantagens em termos de latência e throughput. No entanto, esse impacto tende a ser relativo, uma vez que, em muitos sistemas reais, os principais gargalos estão associados ao banco de dados ou à comunicação em rede, e não à lógica interna da aplicação. Essa hipótese será investigada empiricamente neste trabalho (PEREIRA; DINIZ, 2022).

Por fim, a Tabela 1 sintetiza os principais pontos discutidos, apresentando uma visão comparativa entre as arquiteturas com base nos critérios relevantes para este estudo, bem como as referências que fundamentam cada aspecto analisado.

Tabela 1 – Análise comparativa entre as arquiteturas MVC e Hexagonal.

Critério	MVC	Hexagonal	Vantagem	Ref.
Acoplamento	● ● ○	● ○ ○	Hexagonal	^a
Testabilidade	● ● ○	● ● ●	Hexagonal	^b
Substituição de componentes	● ○ ○	● ● ●	Hexagonal	^c
Manutenibilidade	● ● ○	● ● ●	Hexagonal	^d
Simplicidade estrutural	● ● ●	● ● ○	MVC	^e
Desempenho sob carga	● ● ●	● ● ●	-	^f
Adoção / curva de aprendizado	● ● ●	● ○ ○	MVC	^g

● ○ ○ Baixa ● ● ○ Médio ● ● ● Alta

^a (BASS; CLEMENTS; KAZMAN, 2012; MARTIN, 2017; COCKBURN, 2005).

^b (COCKBURN, 2005; MARTIN, 2017).

^c (DAMIR; SERGII, 2020; COCKBURN, 2005).

^d (SOMMERVILLE, 2011; ISO/IEC, 2011).

^e (INTEGRAL.IO, 2024; MIGUEL, 2025).

^f (PEREIRA; DINIZ, 2022); hipótese empírica deste trabalho.

^g (UCHITELLE; THOMAS, 2023; INTEGRAL.IO, 2024).

Fonte: O Autor. (2026)

A Tabela 1 apresenta uma síntese comparativa entre MVC e Arquitetura Hexagonal considerando critérios relacionados aos objetivos deste estudo. A análise evidencia que cada abordagem apresenta diferentes compromissos arquiteturais. Enquanto o MVC prioriza uma estrutura mais simples e direta, favorecendo a implementação inicial e reduzindo a quantidade de abstrações necessárias, a Arquitetura Hexagonal enfatiza a separação de responsabilidades e o isolamento do domínio em relação aos componentes externos.

Os critérios relacionados ao acoplamento, testabilidade, substituição de componentes e manutenibilidade destacam vantagens teóricas da Arquitetura Hexagonal. Isso ocorre devido à utilização de portas e adaptadores, que estabelecem contratos explícitos entre o núcleo da aplicação e elementos de infraestrutura. Como consequência, alterações em mecanismos externos, como bancos de dados ou serviços de terceiros, tendem a produzir menor impacto nas regras centrais do sistema.

Por outro lado, aspectos como simplicidade estrutural, adoção e curva de aprendizado apresentam vantagem para o MVC, principalmente devido à sua ampla utilização e menor quantidade de conceitos adicionais necessários para sua implementação. Essa característica torna o MVC uma alternativa atrativa em aplicações menores, projetos com requisitos mais estáveis ou cenários nos quais a velocidade inicial de desenvolvimento é um fator relevante.

Os critérios de desempenho e manutenibilidade, destacados na tabela, constituem o foco empírico deste trabalho. No caso da manutenibilidade, a literatura sugere que a Arquitetura He-

xagonal apresenta vantagens decorrentes do isolamento do domínio, da redução do acoplamento e da maior facilidade de testes e substituição de componentes. Já em relação ao desempenho, não há consenso que permita afirmar a superioridade de uma abordagem sobre a outra apenas com base na análise teórica. Embora a Arquitetura Hexagonal introduza camadas adicionais de abstração, o impacto efetivo dessas estruturas depende da implementação e do contexto de execução.

Dessa forma, a possível influência da arquitetura sobre métricas como latência e vazão é tratada neste estudo como uma hipótese de pesquisa a ser verificada experimentalmente. A comparação entre as implementações MVC e Hexagonal buscará identificar se as diferenças observadas são significativas e se os benefícios relacionados à manutenibilidade implicam algum custo mensurável em termos de desempenho.

2.5 TRABALHOS RELACIONADOS

A comparação de desempenho entre diferentes abordagens arquiteturais e tecnológicas é um tema recorrente na engenharia de software. De modo geral, muitos estudos seguem uma lógica semelhante à adotada neste trabalho: desenvolvem aplicações funcionalmente equivalentes e as submetem a testes de carga controlados, com o objetivo de coletar métricas objetivas de desempenho. Os trabalhos apresentados a seguir foram selecionados por apresentarem proximidade metodológica com esta pesquisa seja nas ferramentas utilizadas, na forma de construção dos cenários ou no tipo de análise realizada e são organizados em ordem crescente de aderência ao escopo deste estudo.

Durante o levantamento bibliográfico, não foram encontrados estudos que realizassem uma comparação empírica direta entre as arquiteturas MVC e Hexagonal nos moldes aqui propostos, isto é, mantendo constantes o *runtime*, o *framework* HTTP e o banco de dados, e variando apenas a arquitetura. Essa ausência sugere uma possível lacuna na literatura e reforça a relevância deste trabalho. Por esse motivo, os trabalhos relacionados foram selecionados com base na similaridade metodológica, e não necessariamente no mesmo objeto de estudo. Ainda que abordem outros eixos de comparação como *runtimes*, *frameworks* ou estilos arquiteturais distintos, eles contribuem diretamente para a construção do desenho experimental adotado neste trabalho.

Milet (2024) comparou o desempenho dos *runtimes* JavaScript Node.js, Deno e Bun, utilizando como base a metodologia clássica de avaliação de desempenho proposta por (JAIN, 1991). Para isso, o autor desenvolveu três aplicações equivalentes, cada uma com três tipos de endpoints: uma rota simples de resposta estática (/), uma rota com processamento intensivo de CPU (/fibonacci) e uma rota de inserção em banco de dados em memória (/addUser). Os testes foram realizados com a ferramenta Autocannon, considerando dois níveis de carga 10 e 100 conexões simultâneas com duração de dois minutos cada. Foram coletadas métricas como

throughput, latência e número de requisições por segundo, além de dados de uso de CPU e memória via *docker stats*. Os resultados mostraram que Deno e Bun tendem a apresentar melhor desempenho que o Node.js, embora essa vantagem diminua com o aumento da carga. Um ponto que chama atenção é o aumento expressivo da latência de pior caso chegando a até 424 vezes quando a carga cresce, sem uma queda proporcional no throughput, indicando um comportamento de degradação assimétrico. A estratégia metodológica adotada por Milet é bastante próxima da utilizada neste trabalho e fundamenta decisões como a separação dos cenários por tipo de operação e a inclusão de cargas mais elevadas (MILET, 2024).

Santos et al. (2025) realizaram um estudo comparando cinco *frameworks backend* Spring Boot, ASP.NET Core, Express.js, Flask e Actix em um ambiente controlado com containers Docker. Cada aplicação foi limitada a 1 GB de memória e um núcleo de CPU, garantindo condições equivalentes de execução. Os cenários de teste foram variados, incluindo desde operações simples até processamento intensivo, leitura de arquivos e escrita em disco. Para reduzir variações nos resultados, cada teste foi repetido 30 vezes, com reinício dos containers entre execuções. As métricas analisadas incluíram requisições por segundo, consumo de CPU e memória. Os resultados indicaram que não existe uma solução universalmente superior: o desempenho depende do tipo de operação executada. Embora o estudo compare tecnologias diferentes o que dificulta o isolamento de variáveis, duas práticas foram diretamente incorporadas neste trabalho: a divisão dos testes por tipo de operação e a repetição dos experimentos para maior confiabilidade dos resultados (SANTOS *et al.*, 2025).

Lima (2023) realizou um experimento controlado comparando cinco *frameworks web backend*: Django, Spring Boot, Flask, Ruby on Rails e Laravel, avaliando métricas como tempo de resposta, consumo de recursos, taxa de erro e vazão em diferentes níveis de concorrência. Um diferencial do estudo foi a implementação de duas versões da aplicação, uma utilizando ORM e outra com acesso direto ao banco de dados, permitindo analisar o impacto da camada de persistência. Os testes foram conduzidos com Apache JMeter em ambiente controlado, seguindo um protocolo experimental definido. Os resultados indicaram destaque do Spring Boot nas métricas de tempo de resposta e vazão. Embora o estudo avalie desempenho entre tecnologias, esta pesquisa diferencia-se ao analisar os impactos da escolha arquitetural entre MVC e Arquitetura Hexagonal, considerando também aspectos de manutenibilidade (LIMA, 2023).

Pereira e Diniz (2022) compararam arquiteturas monolítica e de microsserviços por meio de sistemas equivalentes em um cenário corporativo. Os testes foram realizados com Autocannon, em diferentes níveis de carga, coletando métricas como latência média, requisições por segundo e distribuição de latência por percentis. Os resultados apontaram melhor desempenho da arquitetura monolítica em todos os cenários, especialmente sob cargas mais elevadas. Os autores também avaliaram aspectos qualitativos, como complexidade de desenvolvimento e testes. Esse trabalho é o mais próximo da proposta desta pesquisa, por comparar estilos arquiteturais e utilizar a mesma ferramenta de *benchmarking* (PEREIRA; DINIZ, 2022).

De forma geral, esta pesquisa se diferencia dos trabalhos analisados em três aspectos principais. Primeiro, pelo controle experimental: ao manter constantes o *runtime*, o *framework* HTTP e o banco de dados, e variar apenas a arquitetura, torna-se possível isolar com maior precisão o impacto arquitetural. Segundo, pela forma de avaliar a manutenibilidade, baseada em métricas objetivas extraídas automaticamente do código, em vez de avaliações subjetivas. Por fim, quanto à análise dos resultados, reconhece-se a possibilidade de incorporar métodos estatísticos mais avançados em trabalhos futuros, conforme demonstrado por (LIMA, 2023). Assim, este estudo se posiciona como uma investigação focada especificamente na comparação arquitetural, complementando pesquisas existentes e contribuindo diretamente para a tomada de decisão em projetos baseados em Node.js, Express e PostgreSQL.

3 METODOLOGIA

Este capítulo apresenta os procedimentos metodológicos adotados para a realização deste trabalho, abrangendo a classificação da pesquisa, a definição do ambiente experimental, as tecnologias e ferramentas utilizadas, os cenários de teste estabelecidos e as métricas coletadas para a comparação entre as arquiteturas MVC e Hexagonal.

3.1 CLASSIFICAÇÃO DA PESQUISA

Este estudo caracteriza-se como uma pesquisa aplicada, de abordagem quantitativa e qualitativa, com delineamento experimental e comparativo. A natureza aplicada se justifica pelo objetivo de gerar dados relacionados com a resolução de um problema prático cujos resultados podem ser relevantes para a escolha arquitetural em aplicações web.

A abordagem quantitativa manifesta-se na coleta e análise de métricas objetivas de desempenho, como *throughput*, latência e taxa de erros, obtidas por meio de testes de carga realizados em ambiente controlado. Já a abordagem qualitativa complementa essa análise ao considerar aspectos relacionados à manutenibilidade, como o nível de acoplamento entre camadas, a testabilidade e a facilidade de substituição de componentes, avaliados a partir da inspeção estruturada do código-fonte das implementações.

Quanto aos objetivos, a pesquisa possui caráter descritivo e explicativo. Descritivo, ao caracterizar o comportamento de cada implementação arquitetural sob condições controladas de execução; e explicativo, ao buscar identificar e analisar relações entre as decisões arquiteturais adotadas e os resultados observados nos experimentos. Para reduzir possíveis ameaças relacionadas à interpretação das arquiteturas, será realizada uma etapa prévia de validação estrutural das implementações, verificando sua aderência aos princípios esperados de cada abordagem, como separação de responsabilidades, direção das dependências e isolamento entre componentes.

3.2 AMBIENTE EXPERIMENTAL

Para assegurar que as diferenças nos experimentos quantitativos sejam atribuídas exclusivamente às decisões arquiteturais, ambas as implementações utilizarão o mesmo conjunto de tecnologias, composto por Node.js, Express e PostgreSQL. As duas APIs serão desenvolvidas com paridade funcional completa, expondo os mesmos endpoints e regras de negócio, diferenciando-se apenas na organização interna do código, conforme os princípios das arquiteturas MVC e Hexagonal.

O ambiente de execução será configurado em um único host local, no qual as duas APIs

e o banco de dados serão executados simultaneamente. Essa abordagem tem como objetivo minimizar a influência de variáveis externas, como latência de rede entre serviços ou variações de infraestrutura distribuída, garantindo que os resultados obtidos reflitam, de forma mais precisa, os impactos das escolhas arquiteturais.

As APIs serão executadas em portas distintas, conforme apresentado na Tabela 2.

Tabela 2 – Discriminação das Portas

Aplicação	Porta
API MVC	3000
API Hexagonal	3001
PostgreSQL	5432

Fonte: O Autor (2026).

3.3 FERRAMENTA DE BENCHMARKING

Os testes de desempenho serão conduzidos utilizando a biblioteca autocannon, uma ferramenta de benchmarking HTTP de alta performance desenvolvida em Node.js. O autocannon opera estabelecendo múltiplas conexões TCP persistentes (*keep-alive*) com a API alvo e enviando requisições de forma contínua durante um intervalo de tempo pré-definido, coletando métricas de desempenho em tempo real.

O principal critério para a escolha do autocannon é a compatibilidade de *runtime*. Por ser desenvolvido na mesma plataforma das APIs avaliadas (Node.js), a ferramenta reduz potenciais interferências relacionadas à interoperabilidade entre diferentes ambientes de execução. Além disso, o autocannon é amplamente utilizado em *benchmarks* oficiais de *frameworks* como o Fastify, o que reforça sua confiabilidade para esse tipo de análise (FASTIFY, 2024).

Essa escolha também encontra respaldo na literatura. Estudos como os de Milet (2024) e Pereira e Diniz (2022) adotam o autocannon em experimentos comparativos semelhantes, evidenciando sua adequação para a realização de testes de carga em aplicações web baseadas em Node.js.

3.4 CENÁRIOS DE TESTE

Serão executados três cenários de teste, cada um voltado à avaliação de um aspecto específico do comportamento das APIs sob carga. A separação dos cenários por tipo de operação segue práticas amplamente adotadas em estudos comparativos de desempenho, nos quais diferentes perfis de carga são analisados de forma isolada. Essa abordagem reduz a interferência entre variáveis e permite uma avaliação mais precisa do comportamento do sistema (JAIN, 1991; MILET, 2024; SANTOS *et al.*, 2025).

3.4.1 Leitura Pura

Neste cenário, todas as conexões realizam requisições contínuas do tipo GET. O objetivo é medir o desempenho da camada de consulta ao banco de dados e da serialização dos dados. Por ser um cenário mais simples e previsível, é esperado que apresente o maior *throughput* entre os três.

3.4.2 Escrita Pura

Aqui, todas as conexões executam requisições POST com corpo em JSON. Para evitar conflitos causados pela regra de e-mail único que poderiam distorcer as métricas é criado previamente um conjunto de e-mails únicos utilizando UUIDs (*Universally Unique Identifier*), um valor de 128 bits gerado automaticamente e padronizado pela RFC 4122 (LEACH; MEAL-LING; SALZ, 2005). Durante o teste, o autocannon percorre esse conjunto, garantindo que cada requisição utilize um e-mail diferente. Esse cenário tende a ser mais lento que o de leitura, pois envolve validações adicionais, inserção no banco e serialização dos dados.

3.4.3 Carga Mista

Este cenário busca simular um uso mais próximo do real, em que operações de leitura ocorrem com mais frequência do que escritas. Para isso, são utilizadas duas instâncias do autocannon executando em paralelo, com a distribuição das conexões definida conforme apresentado na Tabela 3.

Tabela 3 – Distribuição de conexões no cenário misto

Instância	Endpoint	Proporção
autocannon A	GET	70% das conexões
autocannon B	POST	30% das conexões

Fonte: O Autor (2026).

As métricas extraídas serão detalhadas nos próximos capítulos. Ao final de cada execução do cenário misto, os resultados obtidos pelas duas instâncias são consolidados. O *throughput* total corresponde à soma dos valores coletados em ambas as aplicações, enquanto a latência é representada pelo pior caso observado entre elas. Já a latência média é calculada por meio de uma média ponderada pelo número de conexões, conforme apresentado por (MILET, 2024).

3.5 PARÂMETROS E NÍVEIS DE CARGA

Foram estabelecidos dois cenários distintos, caracterizados pelo número de conexões simultâneas: 10 e 50, conforme a Tabela 4, permitindo simular diferentes níveis de concorrência entre usuários. Em cada cenário, requisições HTTP dos tipos GET e/ou POST serão enviadas

continuamente a um endpoint específico de cada implementação durante um período de 2 minutos. Como a carga é definida em função do tempo de execução, o número total de requisições processadas varia conforme a capacidade de processamento de cada implementação.

Tabela 4 – Níveis de carga definidos para os testes

Nível	Conexões Simultâneas	Finalidade
Leve	10	Verificação de sanidade e baseline
Médio	50	Resultado principal carga moderada e estável

Fonte: O Autor (2026).

O nível médio de 50 conexões simultâneas será adotado como resultado principal para fins de comparação entre as arquiteturas.

3.6 MÉTRICAS

Para cada cenário e nível de carga, serão coletadas as métricas apresentadas na Tabela 5. Essas métricas foram escolhidas por serem amplamente utilizadas na avaliação de desempenho de aplicações web, pois permitem observar diferentes dimensões do comportamento do sistema, como capacidade de processamento, tempo de resposta e confiabilidade.

A inclusão de métricas como *throughput*, latência média, percentis e taxa de erros está alinhada às recomendações clássicas para avaliação de desempenho de sistemas computacionais, que destacam a importância de analisar, de forma conjunta, a capacidade, a responsividade e a confiabilidade do sistema (JAIN, 1991). Além disso, o uso de métricas baseadas em percentis é relevante por evidenciar a experiência percebida pelos usuários em situações mais próximas do pior caso (PEREIRA; DINIZ, 2022; MILET, 2024).

Tabela 5 – Métricas coletadas nos testes de desempenho

Métrica	Descrição
Throughput (req/s)	Número de requisições processadas com sucesso por segundo
Latência média	Tempo médio de resposta das requisições
p50 (mediana)	Valor abaixo do qual 50% das latências se encontram
p95	Valor abaixo do qual 95% das latências se encontram
p99	Valor abaixo do qual 99% das latências se encontram
Taxa de erros	Proporção de requisições que resultaram em falha

Fonte: O Autor (2026).

O *throughput* é adotado como a principal métrica de capacidade, pois representa diretamente o volume de requisições que o sistema consegue processar em um determinado intervalo de tempo. Dessa forma, ele permite observar como a aplicação se comporta à medida que a carga aumenta, sendo essencial para a análise de escalabilidade.

A latência média, por sua vez, oferece uma visão geral do desempenho, mas deve ser interpretada com cautela, já que pode ser distorcida por valores extremos (*outliers*). Por isso, sua

análise é complementada pelo uso de percentis, que fornecem uma leitura mais representativa da experiência real dos usuários.

Nesse contexto, os percentis especialmente p95 e p99 ganham destaque, pois evidenciam o comportamento do sistema em cenários próximos ao pior caso. Esses indicadores são amplamente utilizados na literatura justamente por capturarem degradações de desempenho que passam despercebidas quando se considera apenas a média.

A taxa de erros é empregada como uma métrica de confiabilidade, indicando em que medida o sistema consegue responder adequadamente às requisições. Valores elevados costumam sinalizar problemas como saturação de recursos ou falhas na aplicação, sendo, portanto, um aspecto crítico na avaliação sob carga.

Em conjunto, essas métricas permitem uma análise mais completa do sistema, considerando não apenas sua capacidade de processamento, mas também sua estabilidade e a qualidade da experiência percebida pelos usuários.

3.6.1 Exemplo de cálculo das métricas

Para melhor compreensão das métricas adotadas, apresenta-se a seguir um exemplo simplificado de como cada indicador é calculado a partir dos dados coletados durante a execução de um teste de carga.

Considere um cenário hipotético no qual foram realizadas 10 requisições, com os seguintes tempos de resposta (em milissegundos):

[12, 15, 18, 20, 22, 25, 30, 40, 80, 120]

Latência média: é obtida pela média aritmética dos tempos de resposta:

$$\frac{[12 + 15 + 18 + 20 + 22 + 25 + 30 + 40 + 80 + 120]}{10} = 38,2 \text{ ms} \quad (3.1)$$

Esse valor fornece uma visão geral do desempenho, mas, isoladamente, não representa toda a variabilidade dos dados.

Percentis: para o cálculo dos percentis, os tempos de resposta devem estar ordenados em ordem crescente (como já apresentado). A partir dessa ordenação, identificam-se os valores que delimitam determinadas proporções da amostra, permitindo analisar como a latência se distribui ao longo das requisições.

Os percentis foram obtidos por interpolação linear sobre os valores ordenados da amostra. Para um conjunto de n observações ordenadas $x_1 \leq x_2 \leq \dots \leq x_n$, a posição correspondente ao percentil P é calculada por:

$$pos = \frac{P}{100} \times (n - 1) + 1 \quad (3.2)$$

Quando pos não é um número inteiro, o valor do percentil é obtido pela interpolação linear entre os elementos adjacentes $x_{\lfloor pos \rfloor}$ e $x_{\lceil pos \rceil}$:

$$V_P = x_{\lfloor pos \rfloor} + (pos - \lfloor pos \rfloor) \times (x_{\lceil pos \rceil} - x_{\lfloor pos \rfloor}) \quad (3.3)$$

onde $\lfloor pos \rfloor$ e $\lceil pos \rceil$ representam, respectivamente, o piso e o teto da posição calculada.

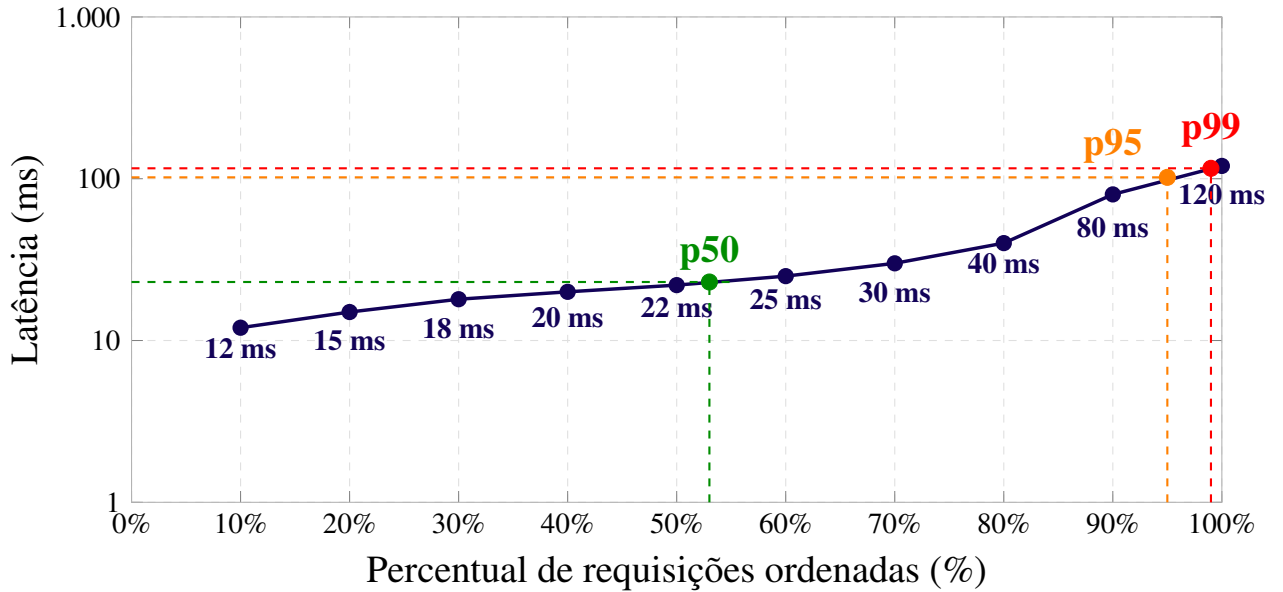
Aplicando ao exemplo, para $n = 10$ observações e $P = 50$:

$$pos = \frac{50}{100} \times (10 - 1) + 1 = 5,5 \quad (3.4)$$

Como $pos = 5,5$, interpola-se entre o quinto elemento ($x_5 = 22$ ms) e o sexto ($x_6 = 25$ ms):

$$V_{50} = 22 + 0,5 \times (25 - 22) = 23,5 \text{ ms} \quad (3.5)$$

Figura 4 – Distribuição de latência com destaque para os percentis p50, p95 e p99



Fonte: O Autor (2026).

A Figura 4 apresenta os tempos de resposta de 10 requisições organizados em ordem crescente. A partir dessa distribuição, observa-se que o percentil p50 é de 23,5 ms, indicando que aproximadamente 50% das requisições foram concluídas em até esse tempo. O percentil p95 situa-se próximo de 102 ms, evidenciando que cerca de 95% das requisições apresentaram latência inferior a esse valor. Já o p99 aproxima-se de 116 ms, indicando que praticamente todas as requisições foram atendidas dentro desse limite.

Throughput (req/s): considerando que as 10 requisições foram concluídas em 2 segundos:

$$\text{Throughput} = \frac{10}{2} = 5 \text{ req/s} \quad (3.6)$$

Taxa de erros: supondo que, das 10 requisições, 2 tenham falhado:

$$\text{Taxa de erros} = \frac{2}{10} = 0,2 \text{ (20\%)} \quad (3.7)$$

Esse conjunto de cálculos demonstra como diferentes métricas capturam aspectos distintos do desempenho. Enquanto o *throughput* indica a capacidade de processamento, as métricas de latência e percentis evidenciam a qualidade da resposta percebida pelos usuários, e a taxa de erros reflete a estabilidade do sistema sob carga.

3.7 AVALIAÇÃO DE MANUTENIBILIDADE

A avaliação da manutenibilidade exige uma abordagem complementar à análise quantitativa de desempenho apresentada nas Seções 3.3 a 3.6. De acordo com a norma ISO/IEC 25010 (ISO/IEC, 2011), essa característica é composta por subcaracterísticas como modularidade, reusabilidade, analisabilidade, modificabilidade e testabilidade. Diferentemente das métricas de desempenho, esses aspectos não podem ser observados apenas por meio de testes de carga, tornando necessária a utilização de técnicas como análise estática do código-fonte e experimentos controlados de modificação.

Neste trabalho, a manutenibilidade será analisada a partir de três eixos complementares. Os dois primeiros são apresentados nas Seções 3.7.1 e 3.7.2, enquanto o terceiro corresponde ao experimento de substituição de componentes descrito na Seção 3.7.3. Cada eixo produz métricas objetivas, reproduzíveis e obtidas automaticamente a partir do código-fonte das duas APIs, reduzindo a subjetividade frequentemente associada a avaliações exclusivamente qualitativas.

Essa estrutura de avaliação fundamenta-se na ISO/IEC 25010, que define manutenibilidade como uma característica de qualidade relacionada à facilidade de compreender, modificar, testar e evoluir sistemas de software (ISO/IEC, 2011). Essa perspectiva também está alinhada à literatura clássica de engenharia de software, que relaciona a manutenibilidade à capacidade de adaptação e evolução contínua dos sistemas ao longo do tempo (SOMMERVILLE, 2011).

3.7.1 Análise de Acoplamento

O acoplamento entre módulos será avaliado a partir do grafo de dependências estáticas extraído do código-fonte das duas APIs. Para isso, será utilizada a ferramenta *dependency-cruiser* (SVERWEIJ, 2024), responsável por percorrer as importações dos arquivos TypeScript e

gerar um grafo direcionado no qual os vértices representam módulos e as arestas representam relações de dependência do tipo “importa”.

A escolha dessa ferramenta justifica-se por sua compatibilidade nativa com TypeScript, tecnologia utilizada em ambas as APIs, e pela possibilidade de definir regras arquiteturais explícitas. Com isso, torna-se possível verificar automaticamente restrições estruturais do sistema, como assegurar que o domínio, na arquitetura Hexagonal, permaneça desacoplado de módulos de infraestrutura.

Com base nesse grafo, serão calculadas as métricas *Ca*, *Ce* e *Instability*, propostas por (MARTIN, 2017) em *Clean Architecture*. Essas métricas permitem representar quantitativamente o grau de dependência entre módulos e avaliar se a estrutura do sistema favorece baixo acoplamento e maior estabilidade arquitetural.

A seguir, apresentam-se as métricas utilizadas nesta análise:

- *Afferent Coupling* (*Ca*): número de módulos que dependem de um dado módulo. Indica o impacto que uma alteração nesse módulo produz no restante do sistema.
- *Efferent Coupling* (*Ce*): número de módulos dos quais o módulo analisado depende. Indica sua exposição a mudanças externas.
- *Instability* (*I*): $I = Ce \div (Ca + Ce)$, variando de 0 (módulo totalmente estável, muito referenciado e com poucas dependências) a 1 (módulo totalmente instável, isolado mas dependente de terceiros).

Segundo (MARTIN, 2017), uma arquitetura bem modularizada tende a apresentar um gradiente de estabilidade: módulos de mais alto nível, como aqueles que concentram regras de negócio, devem apresentar valores de *I* próximos de 0, enquanto módulos de baixo nível, como adaptadores e componentes de infraestrutura, tendem a apresentar valores próximos de 1. A inversão desse gradiente pode indicar uma violação do Princípio da Inversão de Dependência. Nesse sentido, a análise proposta busca verificar se esse comportamento é observado no projeto baseado em Arquitetura Hexagonal e em que medida ele também se manifesta ou não no projeto MVC.

De forma complementar, serão considerados três indicadores estruturais adicionais, que funcionam como critérios de validação da implementação antes da execução dos experimentos: (i) o número de ciclos de dependência, cujo valor esperado é preferencialmente zero, uma vez que ciclos podem aumentar o acoplamento e dificultar o teste isolado dos módulos; (ii) o número de módulos órfãos, ou seja, componentes que não são importados por nenhum outro módulo, podendo indicar código não utilizado, estrutura incompleta ou necessidade de revisão; e (iii) o número de violações de regras de camada, configuradas especialmente no projeto Hexagonal para detectar dependências indevidas do domínio ou dos casos de uso em relação à camada de infraestrutura.

Caso sejam identificadas ocorrências incompatíveis com os princípios arquiteturais adotados, a implementação correspondente deverá ser revisada antes do início dos experimentos de desempenho e manutenibilidade. Essa etapa visa garantir que os resultados obtidos reflitam diferenças relacionadas às características das arquiteturas avaliadas, e não efeitos decorrentes de falhas na organização estrutural do código.

3.7.2 Análise de Testabilidade

A testabilidade é um atributo diretamente relacionado à manutenibilidade, uma vez que sistemas mais fáceis de testar tendem a apresentar menor custo de evolução e maior confiabilidade durante modificações (ISO/IEC, 2011; SOMMERVILLE, 2011). No contexto da Arquitetura Hexagonal, essa característica é reforçada pelo isolamento do domínio e pelo uso de portas e adaptadores, que favorecem a substituição de dependências externas durante os testes (COCKBURN, 2005; MARTIN, 2017).

Neste trabalho, a testabilidade será avaliada a partir de três subindicadores complementares, todos derivados da suíte de testes unitários de cada projeto e executados com a ferramenta Jest (TEAM, 2025):

a) Cobertura de testes por camada. A cobertura será obtida por meio do parâmetro `-coverage` do Jest, configurado para considerar todos os arquivos da pasta `src/`, e não apenas aqueles efetivamente exercitados pelos testes. Essa configuração é essencial para garantir uma comparação justa entre as arquiteturas, pois evita que arquivos sem testes como controllers ou adaptadores sejam excluídos do cálculo, o que inflaria artificialmente os resultados. Os valores de cobertura serão posteriormente agrupados por camada arquitetural, permitindo identificar onde a testabilidade está mais concentrada dentro da aplicação.

b) Tempo total de execução da suíte de testes. Será medido em milissegundos. Em geral, suítes de teste que dependem de infraestrutura real, como banco de dados ou rede, tendem a apresentar tempos significativamente maiores do que aquelas baseadas em simulações em memória. Como, neste trabalho, ambas as APIs serão projetadas para permitir testes unitários sem dependências externas, a diferença observada refletirá principalmente o custo estrutural de cada arquitetura como o uso de *mocks* versus a construção direta de *fakes* a partir de interfaces.

c) Proporção entre *mocks* de módulo e *fakes* de porta. Serão analisadas as ocorrências de `jest.mock(caminho)` e `jest.fn()` em cada projeto:

- `jest.mock(caminho)` intercepta um módulo interno e o substitui por uma implementação controlada. Embora seja uma abordagem funcional, ela acopla os testes a detalhes internos da implementação (por exemplo, ao uso de um método específico como `pool.query` (método usado para executar comandos SQL contra o PostgreSQL)). Como consequência, testes desse tipo podem falhar mesmo quando o contrato externo permanece inalterado.

- `jest.fn()` permite criar funções simuladas sem interceptar módulos, sendo geralmente utilizada para compor objetos que implementam interfaces públicas ou seja, as chamadas *ports*, no contexto da Arquitetura Hexagonal. Essa abordagem mantém os testes desacoplados da implementação e é considerada mais alinhada às boas práticas descritas por (COCKBURN, 2005; MARTIN, 2017).

A proporção entre o uso de `jest.mock` e o total de simulações será utilizada como um indicador do grau de acoplamento dos testes à implementação. Valores elevados tendem a indicar ausência de contratos bem definidos, enquanto valores mais baixos sugerem maior aderência ao Princípio da Inversão de Dependência (2.3.2), com interfaces explícitas facilitando a substituição de componentes.

3.7.3 Substituição de Componentes

A substituição controlada de componentes permite observar empiricamente a modificabilidade da arquitetura, uma das subcaracterísticas de manutenibilidade previstas pela ISO/IEC 25010 (ISO/IEC, 2011). Esse tipo de experimento também está alinhado ao princípio da inversão de dependência, segundo o qual componentes de alto nível devem permanecer independentes de detalhes de infraestrutura (MARTIN, 2017).

O terceiro eixo de avaliação da manutenibilidade consiste na realização de experimentos controlados de substituição de elementos de infraestrutura. Diferentemente das métricas estáticas apresentadas na Seção 3.7, esse experimento busca medir diretamente o esforço necessário para modificar o sistema, atributo central da análise de manutenibilidade proposta neste estudo.

Serão conduzidos três cenários em *branches* isoladas de cada projeto, seguindo exatamente o mesmo procedimento em ambas as arquiteturas, de modo a garantir comparabilidade entre os resultados:

Cenário A - Substituição do mecanismo de persistência. O PostgreSQL será substituído por um repositório em memória. Em ambos os projetos, a paridade funcional deverá ser mantida: os cinco *endpoints* existentes devem continuar respondendo com os mesmos códigos de status e formatos de resposta, e a suíte de testes deve permanecer íntegra, sem alterações nos arquivos `*.test.ts`.

Cenário B - Substituição do framework HTTP. O framework Express será substituído pelo Fastify, preservando os mesmos *endpoints* e mantendo a suíte de testes inalterada.

Cenário C - *Change amplification*. Será adicionado um novo atributo `phone` à entidade `User`, propagando essa alteração ao longo do modelo, dos *endpoints* e dos testes. Esse cenário avalia o custo de evolução do sistema. Enquanto os cenários A e B tendem a favorecer arquiteturas com maior isolamento entre regras de negócio e infraestrutura (hipótese associada à Arquitetura Hexagonal), o cenário C tende a favorecer estruturas com menor quantidade

de abstrações e pontos de propagação de alteração (hipótese associada à implementação MVC adotada neste trabalho). A inclusão desse terceiro cenário é intencional e visa explicitar esse trade-off de forma empírica, reduzindo o risco de viés na análise.

Para cada combinação de cenário e arquitetura, serão coletadas as métricas apresentadas na Tabela 6, obtidas a partir da saída do comando *git diff* (ferramenta do sistema de controle de versão Git que compara o estado atual dos arquivos com uma versão anterior, exibindo as linhas adicionadas e removidas em cada arquivo modificado (CHACON; STRAUB, 2014)) ao final de cada modificação:

Tabela 6 – Métricas que serão coletadas no experimento de substituição.

Métrica	Instrumento
Arquivos criados	<i>git diff --stat</i>
Arquivos modificados	<i>git diff --stat</i>
Linhas adicionadas	<i>git diff --shortstat</i>
Linhas removidas	<i>git diff --shortstat</i>
Testes unitários quebrados	Diferença da suíte pré/pós
Tempo de implementação (min)	Cronômetro, indicador auxiliar

Fonte: O Autor (2026).

As métricas extraídas via *git diff* constituem o principal indicador de análise, por serem objetivas, reproduzíveis e independentes do executor. Já o tempo de implementação será considerado apenas como um indicador auxiliar, uma vez que pode ser influenciado por fatores como familiaridade com o código e curva de aprendizado, especialmente em um experimento conduzido por um único desenvolvedor.

3.8 LIMITAÇÕES

Uma possível ameaça deste estudo está relacionada ao fato de que as implementações a serem desenvolvidas podem não representar, de forma unânime, as arquiteturas MVC e Hexagonal em sua forma “ideal”. Isso ocorre porque tais padrões não são especificações rígidas, mas modelos conceituais amplamente difundidos na literatura. Dessa forma, existem diferentes interpretações possíveis para sua aplicação prática, desde que seus princípios fundamentais sejam respeitados. Assim, as versões que serão implementadas podem ser compreendidas como uma entre várias formas válidas de representação dessas arquiteturas, e não como definições únicas ou definitivas.

Para lidar com essa limitação, algumas medidas serão adotadas durante o desenvolvimento do trabalho.

As decisões de projeto serão fundamentadas nos princípios propostos por Reenskaug (1979) para o padrão MVC e por Cockburn (2005) para a Arquitetura Hexagonal. No caso do MVC, será adotada uma implementação compatível com os princípios do modelo original,

porém adaptada ao contexto de APIs REST, conforme amplamente empregado em aplicações backend contemporâneas. Essa adaptação contempla, entre outros aspectos, a ausência de uma camada de apresentação HTML e a utilização de uma camada de *Repository* para o acesso à persistência, preservando a separação de responsabilidades característica do padrão. Para a Arquitetura Hexagonal, também serão consideradas as contribuições de Martin (2017) quanto à organização das camadas de domínio e aplicação.

A estrutura da implementação Hexagonal será verificada de forma automatizada por meio da ferramenta *dependency-cruiser* (SVERWEIJ, 2024). Serão definidas regras de dependência compatíveis com a arquitetura proposta, permitindo verificar se as camadas de domínio permanecem desacopladas da infraestrutura e identificar eventuais violações das restrições arquiteturais estabelecidas. Esses critérios são definidos antes da execução dos experimentos, e qualquer desvio identificado é corrigido antes das medições.

A implementação MVC seguirá uma estrutura comum em APIs REST desenvolvidas com Node.js e Express, na qual a camada de apresentação é representada pela serialização em JSON e o acesso a dados é organizado em uma camada de repositório separada, mantendo a separação de responsabilidades.

Mesmo com essas medidas, ainda poderá existir certo grau de interpretação na avaliação da aderência arquitetural. Isso é esperado em estudos envolvendo padrões arquiteturais, uma vez que diferentes desenvolvedores podem adotar soluções distintas e ainda assim permanecer alinhados aos mesmos princípios gerais.

4 APLICAÇÃO EXPERIMENTAL

Este capítulo apresenta a aplicação proposta para viabilizar a comparação entre as arquiteturas MVC e Hexagonal neste trabalho. São descritos os critérios de escolha do domínio, o modelo de dados, as regras de negócio e os endpoints da API. Também são detalhadas a estrutura interna de cada implementação e o esquema do banco de dados. Por fim, é apresentada a relação entre os elementos da aplicação e os experimentos definidos na metodologia, evidenciando como o sistema foi planejado para atender aos objetivos da pesquisa.

4.1 JUSTIFICATIVA DA ESCOLHA DO DOMÍNIO

A escolha do domínio da aplicação experimental exige um equilíbrio entre simplicidade e representatividade. Um sistema muito simples tende a não evidenciar diferenças relevantes entre as arquiteturas analisadas, enquanto um sistema excessivamente complexo pode extrapolar o escopo do trabalho e dificultar a manutenção da paridade funcional entre as implementações.

Diante disso, foi adotado um sistema de gerenciamento de tarefas organizado em quadros, inspirado em ferramentas amplamente utilizadas nesse contexto. O domínio é composto por três entidades principais: usuários, quadros e tarefas, organizadas de forma hierárquica. Além disso, o sistema contempla regras de negócio que vão além de operações básicas de armazenamento e recuperação de dados, tornando-o adequado para a análise proposta.

A escolha desse domínio se justifica por quatro razões. A primeira é a presença de regras de negócio reais. A transição controlada de status das tarefas de pendente para em andamento e, posteriormente, para concluída representa uma regra que pertence efetivamente ao domínio da aplicação, e não à infraestrutura. Isso cria uma diferença estrutural perceptível entre as implementações: na Arquitetura Hexagonal, essa lógica permanece centralizada no núcleo do domínio e pode ser testada de forma isolada;

A segunda razão é a adequação aos cenários de *benchmarking* definidos na metodologia. O endpoint de listagem de tarefas envolve consultas com junção entre tabelas, representando uma carga de leitura mais próxima de aplicações reais. Já o endpoint de criação de tarefas exige validações e inserções no banco de dados, caracterizando uma carga de escrita com complexidade suficiente para evidenciar diferenças entre as arquiteturas.

A terceira razão é a viabilidade do experimento de substituição de componentes. As três entidades do domínio permitem executar, de maneira controlada e com esforço mensurável, os cenários de substituição de persistência, de framework HTTP e de propagação de atributos.

Por fim, a quarta razão é a familiaridade do domínio. Por se tratar de um sistema amplamente conhecido, a aplicação pode ser compreendida e avaliada sem necessidade de contex-

tualização adicional.

4.2 REQUISITOS FUNCIONAIS

Os requisitos funcionais descrevem o que o sistema deve fazer. A Tabela 7 apresenta os seis requisitos funcionais da aplicação, mapeados às regras de negócio e aos cenários de *benchmarking* definidos na metodologia.

Tabela 7 – Requisitos Funcionais

ID	Requisito	Descrição
RF01	Gerenciar usuários	Permitir criar, listar, buscar, atualizar e excluir usuários.
RF02	Gerenciar quadros	Permitir criar, listar, buscar e excluir quadros associados a um usuário.
RF03	Criar tarefas	Permitir criar tarefas com título, descrição, prioridade e responsável opcional.
RF04	Atualizar status de tarefa	Permitir avançar o status na sequência: todo → in_progress → done.
RF05	Listar e filtrar tarefas	Permitir listar tarefas utilizando filtros por quadro.
RF06	Atualizar e excluir tarefas	Permitir atualizar informações e excluir tarefas individualmente.

Fonte: O Autor (2026).

4.3 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais descrevem as propriedades e restrições que as implementações devem satisfazer para garantir a validade do experimento comparativo. A Tabela 8 apresenta os seis requisitos não funcionais da aplicação.

Tabela 8 – Requisitos Não Funcionais

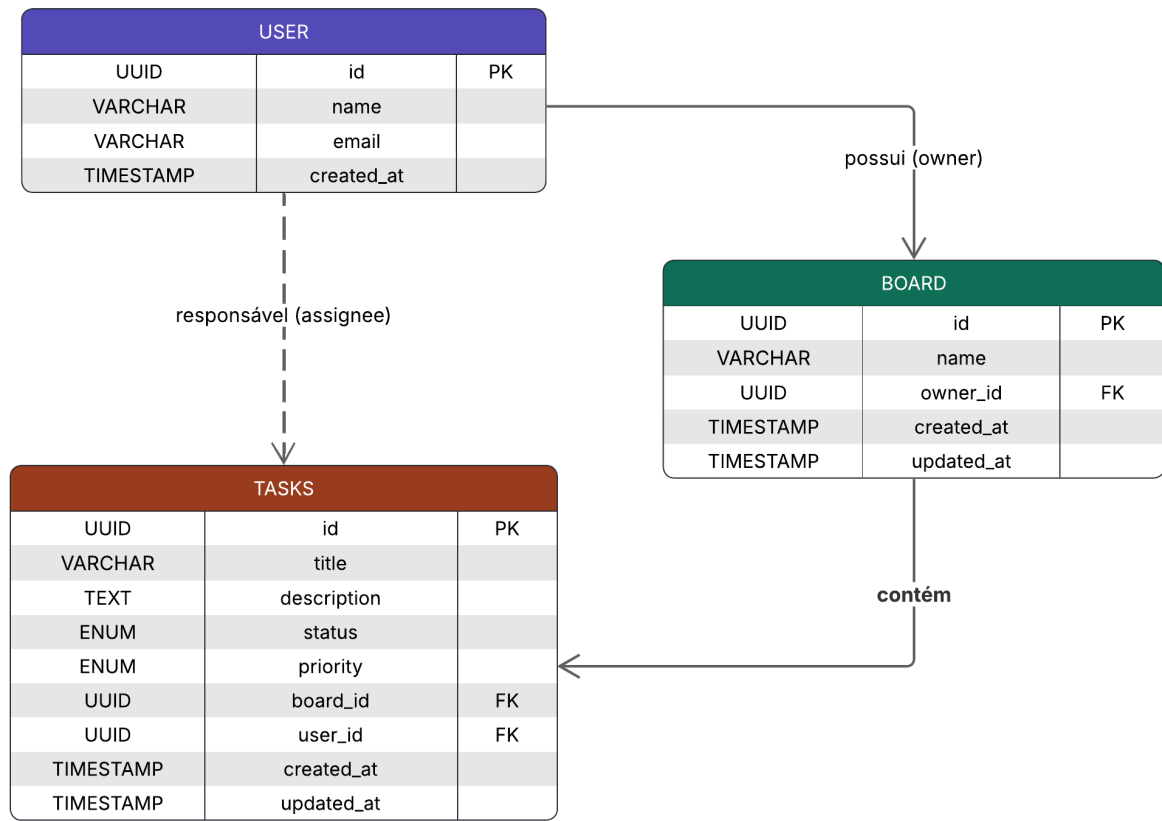
ID	Requisito	Descrição
RNF01	Paridade funcional	As versões MVC e Hexagonal devem expor os mesmos endpoints e códigos HTTP, garantindo comparação justa.
RNF02	Stack tecnológico compartilhado	Ambas as implementações devem utilizar Node.js, Express e PostgreSQL, variando apenas a arquitetura interna.
RNF03	Testabilidade sem banco de dados	A versão Hexagonal deve permitir testes unitários sem dependência de banco real.
RNF04	Desempenho mensurável	As APIs devem suportar os cenários de benchmarking sem erros HTTP nos níveis leve e médio.
RNF05	Isolamento de domínio	A arquitetura Hexagonal não deve permitir dependências do domínio para a infraestrutura.
RNF06	Rastreabilidade por git diff	O repositório deve permitir comparação mensurável das alterações via git diff –stat.

Fonte: O Autor (2026).

4.4 MODELO DE DADOS

A aplicação é estruturada em torno de três entidades principais. A Figura 5 apresenta o Diagrama Entidade-Relacionamento (ER) com as entidades, atributos e relacionamentos do sistema.

Figura 5 – Diagrama Entidade-Relacionamento (ER)



Fonte: O Autor (2026).

4.4.1 Entidade User

A entidade User representa os usuários do sistema. Seus atributos estão descritos na Tabela 9.

Tabela 9 – Atributos da entidade User

Atributo	Tipo	Restrições
id	UUID	Chave primária; gerado automaticamente
name	VARCHAR(255)	Obrigatório; mínimo de 2 caracteres
email	VARCHAR(255)	Obrigatório; único no sistema; formato válido
created_at	TIMESTAMP	Gerado automaticamente na inserção

Fonte: O Autor (2026).

4.4.2 Entidade Board

A entidade Board representa os quadros de tarefas. Cada quadro pertence a um usuário, identificado pelo atributo owner_id. Seus atributos estão descritos na Tabela 10.

Tabela 10 – Atributos da entidade Board

Atributo	Tipo	Restrições
id	UUID	Chave primária; gerado automaticamente
name	VARCHAR(255)	Obrigatório; mínimo de 3 caracteres
owner_id	UUID	Obrigatório; chave estrangeira para User
created_at	TIMESTAMP	Gerado automaticamente na inserção

Fonte: O Autor (2026).

4.4.3 Entidade Task

A entidade Task representa as tarefas criadas dentro de um quadro. É a entidade central da aplicação, pois concentra as principais regras de negócio e é o alvo dos cenários de benchmarking. Seus atributos estão descritos na Tabela 11.

Tabela 11 – Atributos da entidade Task

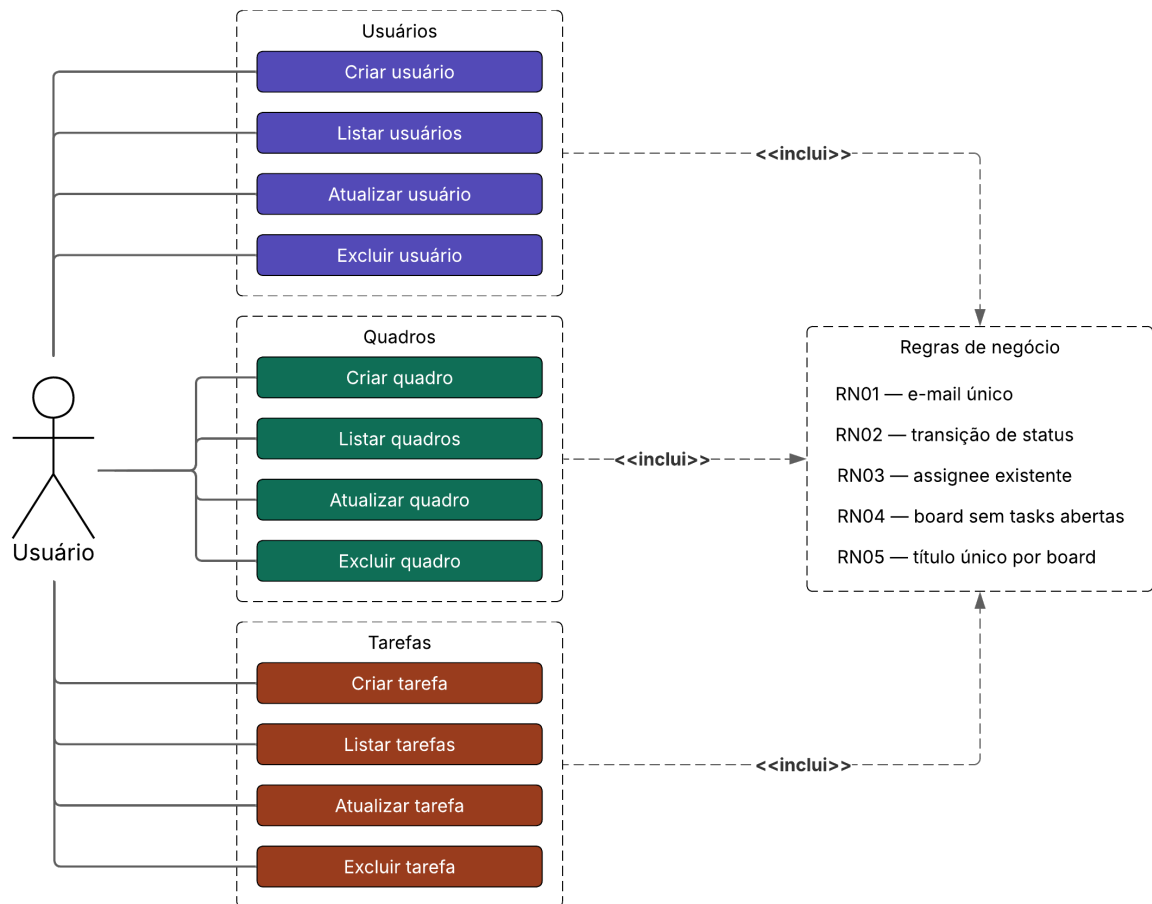
Atributo	Tipo	Restrições
id	UUID	Chave primária; gerado automaticamente
title	VARCHAR(255)	Obrigatório; mínimo de 3 caracteres; único por Board
description	TEXT	Opcional
status	ENUM	Valores: todo in_progress done; padrão: todo
priority	ENUM	Valores: low medium high; padrão: medium
board_id	UUID	Obrigatório; chave estrangeira para Board
assignee_id	UUID	Opcional; chave estrangeira para User
created_at	TIMESTAMP	Gerado automaticamente na inserção
updated_at	TIMESTAMP	Atualizado automaticamente a cada modificação

Fonte: O Autor (2026).

4.5 CASOS DE USO

A Figura 6 apresenta o Diagrama de Casos de Uso da aplicação, evidenciando o único ator do sistema o Usuário e os doze casos de uso distribuídos em três grupos funcionais: Usuários, Quadros e Tarefas.

Figura 6 – Diagrama de Casos de Uso



Fonte: O Autor (2026).

4.6 REGRAS DE NEGÓCIO

As cinco regras de negócio a seguir são implementadas de forma idêntica nas duas versões da aplicação, garantindo a paridade funcional exigida pelo experimento. A diferença entre as arquiteturas não está no que as regras fazem, mas em onde e como elas são implementadas no código.

- RN01 – Unicidade de e-mail: não podem coexistir dois usuários com o mesmo endereço de e-mail no sistema.
- RN02 – Transição controlada de status: o status de uma tarefa só pode avançar na sequência *todo* → *in_progress* → *done*. Retrocessos não são permitidos. Esta é a regra de negócio central do experimento: na Arquitetura Hexagonal, ela é encapsulada no método da entidade Task e testada sem dependência de banco de dados; no MVC, ela tende a residir no Controller sem separação formal.

- RN03 – Pertencimento do responsável: o usuário atribuído como responsável por uma tarefa (assignee) deve existir no sistema.
- RN04 – Restrição de exclusão de quadro: um *Board* não pode ser excluído enquanto possuir tarefas com status *todo* ou *in_progress*.
- RN05 – Unicidade de título por quadro: não podem existir duas tarefas com o mesmo título dentro do mesmo *Board*.

4.7 ENDPOINTS DA API

As duas implementações expõem exatamente os mesmos quinze endpoints, organizados em três recursos. A paridade de rotas, códigos de status HTTP e formatos de resposta é condição necessária para a validade do experimento comparativo.

4.7.1 Recurso Users

Tabela 12 – Endpoints do recurso Users

Método	Rota	Descrição	Regras	Sucesso
POST	/users	Criar usuário	RN01	201
GET	/users	Listar usuários	-	200
GET	/users/:id	Buscar usuário por ID	-	200
PUT	/users/:id	Atualizar usuário	RN01	200
DELETE	/users/:id	Excluir usuário	-	204

Fonte: O Autor (2026).

4.7.2 Recurso Boards

Tabela 13 – Endpoints do recurso Boards

Método	Rota	Descrição	Regras	Sucesso
POST	/boards	Criar quadro	—	201
GET	/boards	Listar quadros	—	200
GET	/boards/:id	Buscar quadro por ID	—	200
DELETE	/boards/:id	Excluir quadro	RN04	204

Fonte: O Autor (2026).

4.7.3 Recurso Tasks

Tabela 14 – Endpoints do recurso Tasks

Método	Rota	Descrição	Regras	Sucesso
POST	/tasks	Criar tarefa	RN03, RN05	201
GET	/tasks	Listar tarefas (filtro: board_id)	—	200
GET	/tasks/:id	Buscar tarefa por ID	—	200
PATCH	/tasks/:id/status	Atualizar status	RN02	200
PUT	/tasks/:id	Atualizar tarefa	RN05	200
DELETE	/tasks/:id	Excluir tarefa	—	204

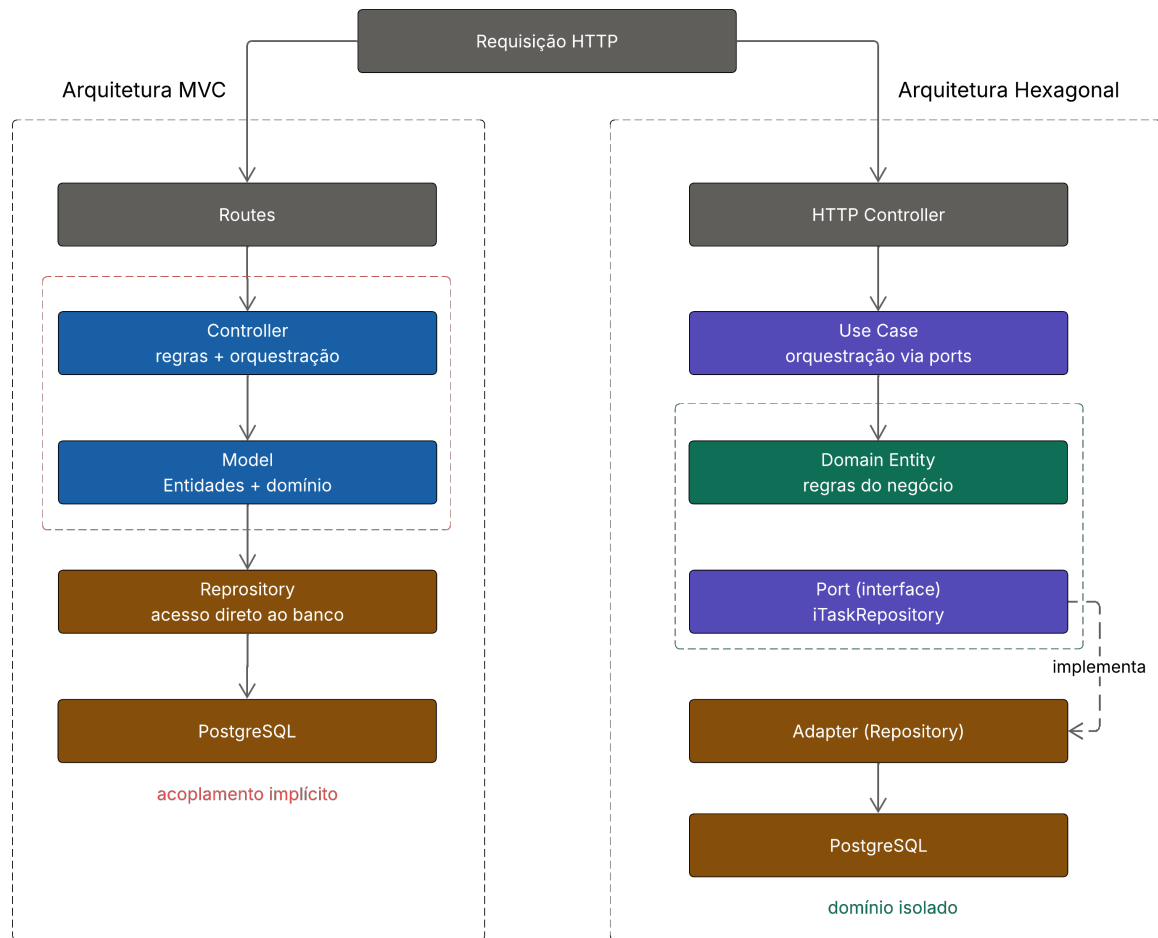
Fonte: O Autor (2026).

4.8 DIAGRAMA DE ARQUITETURA

A Figura 7 apresenta um diagrama comparativo entre as duas implementações, evidenciando o fluxo de uma requisição HTTP em cada arquitetura. No modelo MVC, o fluxo segue o caminho *Routes* → *Controller* → *Model* → *Repository* → *PostgreSQL*, sem a existência de uma fronteira formal entre domínio e infraestrutura. Como consequência, podem surgir dependências implícitas entre essas camadas, aumentando o acoplamento da aplicação.

Já na Arquitetura Hexagonal, o fluxo ocorre por meio de *HTTP Controller* → *Use Case* → *Domain Entity* → *Port (interface)* → *Adapter* → *PostgreSQL*. Nesse modelo, a *Port* atua como uma fronteira explícita entre o núcleo da aplicação e os componentes externos. Isso permite que a infraestrutura implemente contratos definidos pelo domínio, invertendo a direção tradicional das dependências e reduzindo o acoplamento entre as camadas.

Figura 7 – Diagrama de Arquitetura: MVC versus Hexagonal



Fonte: O Autor (2026).

A principal diferença evidenciada pelo diagrama está na localização da lógica de negócio. No MVC, a regra RN02, responsável pela transição de status das tarefas, tende a ficar concentrada no Controller. Isso faz com que a regra dependa diretamente do *framework* HTTP, dificultando sua execução e validação em testes unitários isolados.

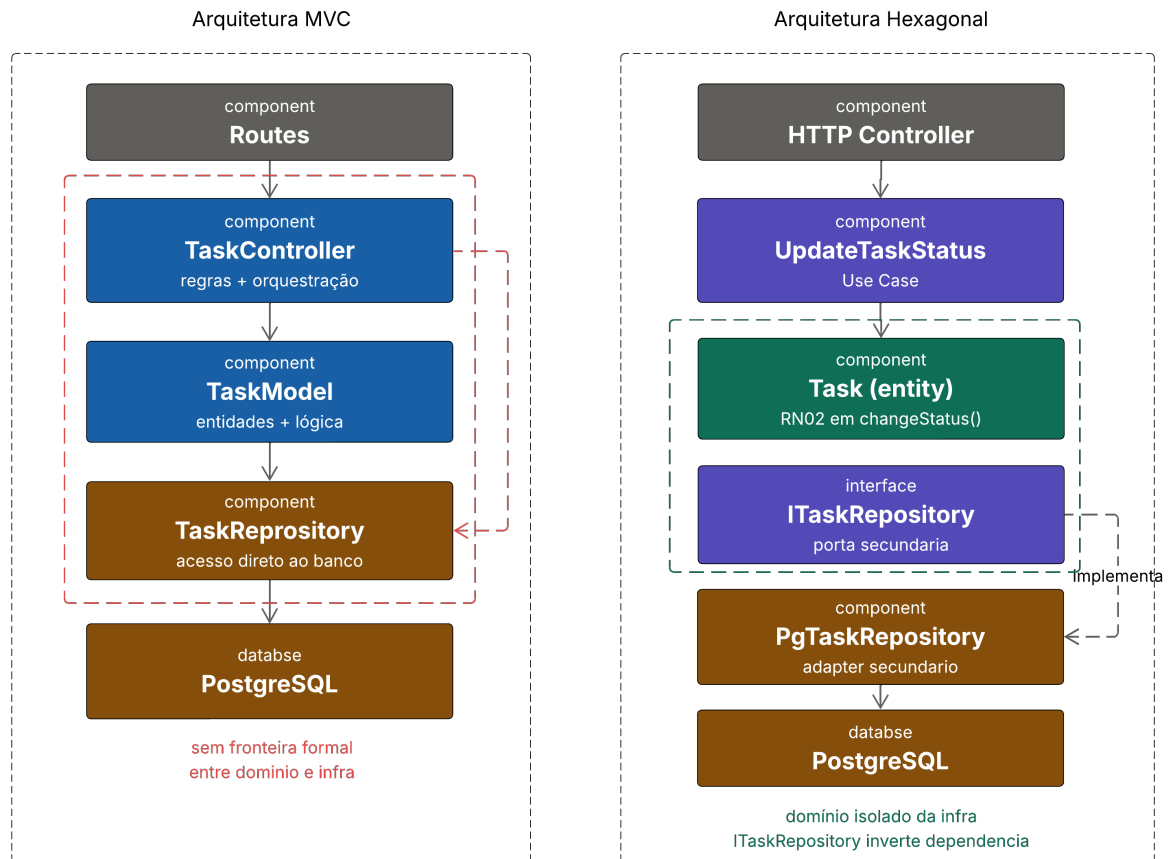
Na Arquitetura Hexagonal, por outro lado, essa mesma regra encontra-se encapsulada na entidade *Task*, dentro do núcleo do domínio. Dessa forma, ela pode ser testada diretamente, sem necessidade de inicializar o servidor HTTP ou estabelecer conexão com o banco de dados, favorecendo o isolamento e a testabilidade da lógica de negócio.

4.9 DIAGRAMA DE COMPONENTES

A Figura 8 apresenta o Diagrama de Componentes das duas implementações, detalhando os componentes de software e suas dependências. No MVC, o componente `TaskController` depende diretamente do componente concreto `TaskRepository`, estabelecendo uma dependência entre a camada de controle e a camada responsável pela persistência. Na Arquitetura He-

xagonal, o componente `Use Case` depende exclusivamente da interface `ITaskRepository`, uma porta secundária da aplicação, enquanto o componente `PgTaskRepository` implementa essa interface, invertendo a direção da dependência e mantendo a lógica de negócio independente da infraestrutura.

Figura 8 – Diagrama de Componentes: MVC versus Hexagonal



Fonte: O Autor (2026).

Essa diferença estrutural é o principal fator que será mensurado nos experimentos de substituição de componentes descritos na Seção 3.7.3. Ao substituir o PostgreSQL por um repositório em memória (Cenário A), a Arquitetura Hexagonal requer apenas a criação de um novo componente que implemente a interface `ITaskRepository`, sem qualquer alteração no domínio ou nos casos de uso. Na implementação MVC adotada, a dependência direta entre o *Controller* e o *Repository* concreto aumenta o acoplamento entre camadas. Como consequência, a substituição de componentes de persistência pode exigir alterações adicionais nos componentes que conhecem diretamente essa implementação.

4.10 ESTRUTURA INTERNA DAS IMPLEMENTAÇÕES

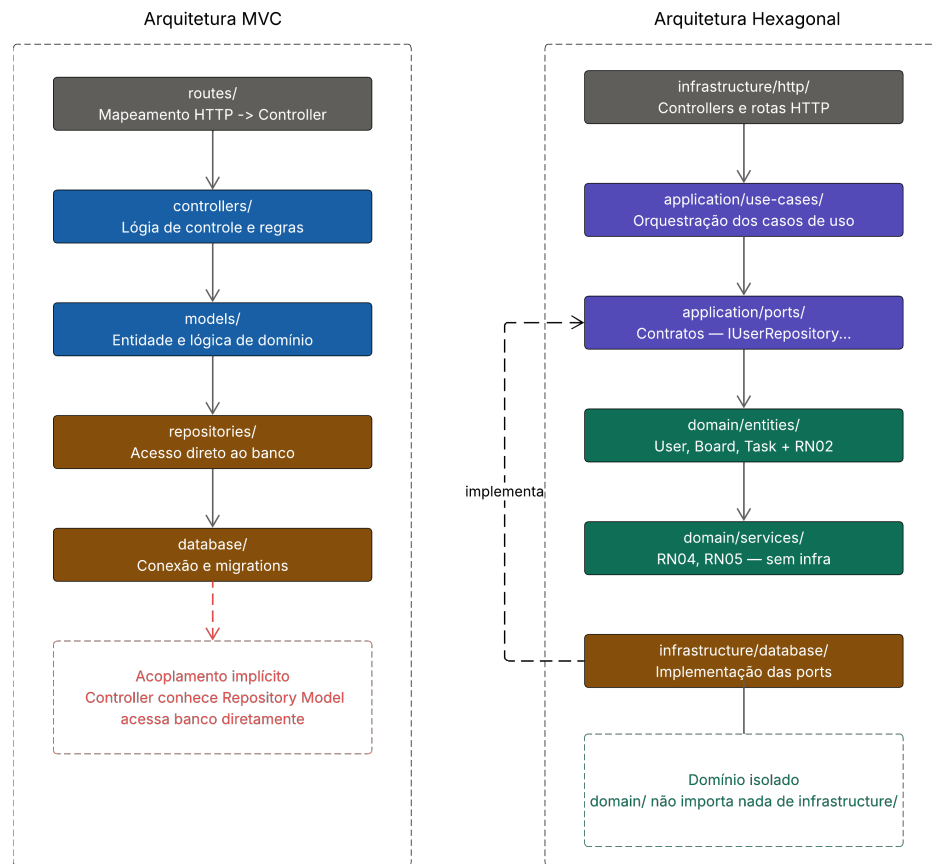
As duas versões da aplicação utilizam o mesmo *stack* tecnológico, diferenciando-se apenas pela forma como os componentes são organizados internamente. A implementação em

MVC foi adaptada ao contexto de uma API REST: como não existe uma camada de apresentação baseada em páginas HTML, a responsabilidade da camada *View* é substituída pela serialização dos dados retornados ao cliente em formato JSON. O acesso ao banco de dados é realizado por meio de uma camada *Repository* independente, seguindo uma organização comum em aplicações desenvolvidas com Node.js e Express (SOMMERVILLE, 2011).

A implementação baseada na Arquitetura Hexagonal segue os princípios definidos por Cockburn (2005), complementados pela organização de camadas proposta por Martin (2017) na *Clean Architecture*. Nesse modelo, a aplicação é estruturada a partir de um núcleo central contendo as regras de negócio, dividido em duas partes principais: *domain/*, responsável pelas entidades e comportamentos do domínio sem dependência de recursos externos, e *application/*, responsável pelos casos de uso e contratos de comunicação (*ports*).

Os elementos externos da aplicação são organizados em *infrastructure/*, onde ficam os adaptadores responsáveis pela comunicação com tecnologias externas, como banco de dados e interfaces HTTP. Esses adaptadores implementam os contratos definidos pelo núcleo da aplicação, permitindo que alterações na infraestrutura ocorram sem modificar diretamente as regras de negócio. Embora Cockburn (2005) não defina uma estrutura interna obrigatória para o núcleo da aplicação, a separação adotada neste trabalho segue uma abordagem amplamente utilizada atualmente, facilitando a organização das responsabilidades e a comparação com a implementação MVC. A Figura 9 apresenta o Diagrama de Pacotes comparativo.

Figura 9 – Diagrama de Pacotes: MVC versus Hexagonal



Fonte: O Autor (2026).

4.10.1 Implementação MVC

A versão MVC organiza o código em camadas horizontais, seguindo a divisão clássica entre *Model*, *View* e *Controller*. As regras de negócio incluindo a RN02 residem nos *Controllers* ou nos *Models*, sem uma interface formal separando o domínio da infraestrutura.

Figura 10 – Estrutura de diretórios da aplicação MVC

```

api-mvc/
src/
  controllers/  -> lógica de controle e regras de negócio
  models/      -> entidades e lógica de domínio
  repositories/ -> acesso ao banco de dados
  routes/      -> mapeamento HTTP -> Controller
  database/    -> conexão e migrations

```

Fonte: O Autor (2026).

4.10.2 Implementação Hexagonal

A versão Hexagonal organiza o código de forma concêntrica, com o domínio no centro e a infraestrutura na periferia. O diretório domain/ não importa nenhum módulo de infrastructure/, restrição verificada automaticamente pelo dependency-cruiser.

Figura 11 – Estrutura de diretórios da aplicação Hexagonal

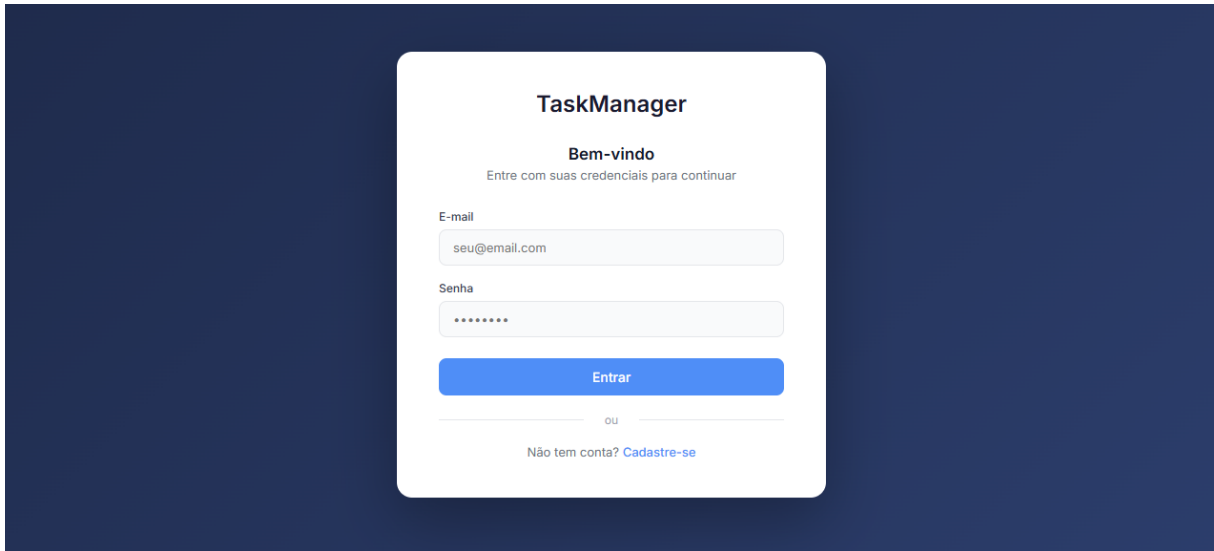
```
api-hexagonal/  
  src/  
    domain/  
      entities/      -> User, Board e Task  
                      (RN02 em changeStatus())  
      value-objects/ -> TaskStatus e Priority  
      services/      -> RN04 e RN05 sem dependência de infra.  
  
    application/  
      use-cases/     -> orquestração dos casos de uso  
      ports/         -> interfaces de repositórios  
  
    infrastructure/  
      http/          -> controllers e rotas  
                      (adaptadores primários)  
      database/      -> implementações concretas das ports
```

Fonte: O Autor (2026).

4.11 INTERFACE DA APLICAÇÃO

A Figura 12 apresenta a tela de autenticação da aplicação. O formulário solicita e-mail e senha, campos que correspondem diretamente ao atributo email da entidade User, sujeito à RN01 de unicidade. A tela representa o ponto de entrada do sistema.

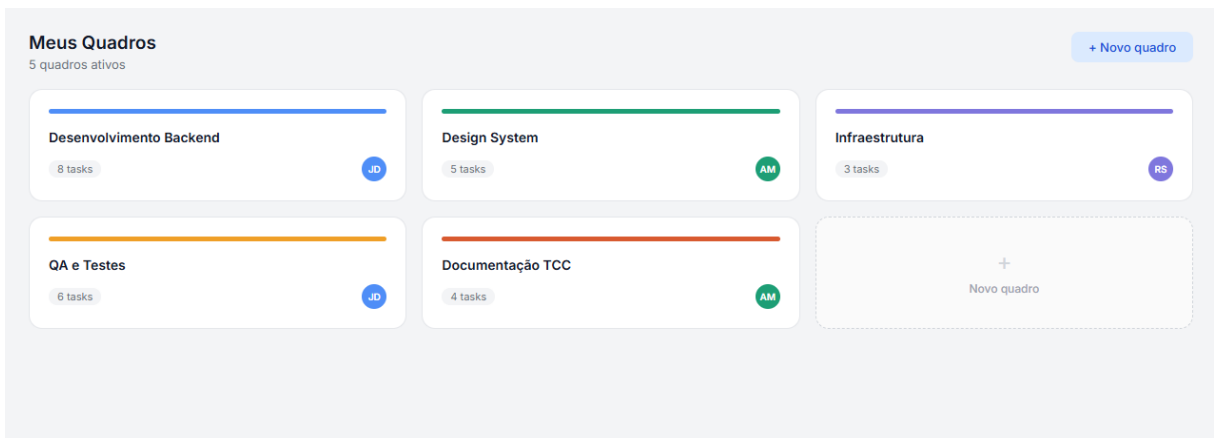
Figura 12 – Tela de Login



Fonte: O Autor (2026).

A Figura 13 ilustra a tela principal da aplicação após o login, exibindo os quadros cadastrados pelo usuário autenticado. Cada cartão apresenta o nome do quadro, a quantidade de tarefas associadas e o avatar do usuário responsável (owner), refletindo o relacionamento entre as entidades *Board* e *User* definido no modelo de dados. A interface corresponde aos endpoints `GET /boards` e é acessada tanto na versão MVC quanto na versão Hexagonal com o mesmo contrato de resposta, garantindo a paridade funcional exigida pelo experimento.

Figura 13 – Tela de listagem de Quadros (Boards)

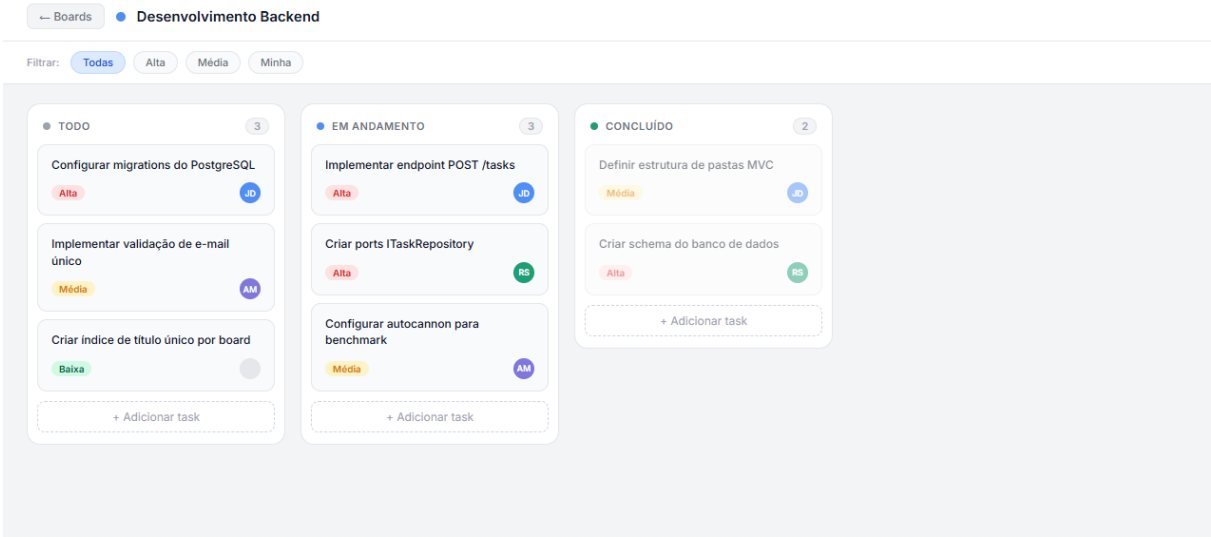


Fonte: O Autor (2026).

A Figura 14 apresenta a tela de gerenciamento de tarefas de um quadro específico, organizada no formato kanban com três colunas: Todo, Em andamento e Concluído. Essa estrutura reflete diretamente os valores do atributo status da entidade Task e a RN02, que restringe a transição entre estados à sequência *todo* → *in_progress* → *done*. Cada cartão exibe o título da

tarefa, o nível de prioridade e o avatar do responsável (*assignee*). As tarefas na coluna Concluído são apresentadas com opacidade reduzida, evidenciando visualmente o estado final do fluxo. Esta tela corresponde ao endpoint GET /tasks com filtro por board_id, utilizado no cenário de Leitura Pura do *benchmarking*.

Figura 14 – Tela de listagem de Tarefas (Tasks)



Fonte: O Autor (2026).

A Figura 15 exibe a tela de gerenciamento de usuários, apresentada em formato de tabela com as colunas nome, e-mail, data de criação e ações disponíveis. Os dados exibidos correspondem aos atributos da entidade User definidos na Tabela 9. A funcionalidade de busca por nome ou e-mail e as ações de edição e exclusão por linha refletem os endpoints GET /users, PUT /users/:id e DELETE /users/:id especificados na Tabela 12. A paginação na parte inferior da tela indica a escalabilidade prevista para o volume de registros.

Figura 15 – Tela de gerenciamento de Usuários (Users)

Usuários			
		Buscar usuário...	+ Novo usuário
USUÁRIO	E-MAIL	CRIADO EM	AÇÕES
João Desenvolvimento	joao@empresa.com	12/01/2026	
Ana Mendes	ana@empresa.com	15/01/2026	
Rafael Santos	rafael@empresa.com	20/01/2026	
Larissa Costa	larissa@empresa.com	03/02/2026	
Marcos Ferreira	marcos@empresa.com	10/02/2026	
Exibindo 5 de 5 usuários Anterior 1 Próxima			

Fonte: O Autor (2026).

5 CONSIDERAÇÕES PARCIAIS

Este trabalho tem como propósito comparar as arquiteturas MVC e Hexagonal no contexto de aplicações web, investigando suas diferenças de desempenho e de manutenibilidade e os contextos em que cada abordagem se mostra mais adequada. Nesta primeira etapa, o foco recaiu sobre a fundamentação teórica e o delineamento do experimento que será conduzido na fase seguinte.

No que se refere aos objetivos específicos, dois foram integralmente alcançados no TCC 1. A caracterização das arquiteturas MVC e Hexagonal foi realizada no Capítulo 2, no qual se descreveram seus componentes, responsabilidades, vantagens e limitações, culminando em uma análise comparativa fundamentada na literatura. O levantamento e a análise de trabalhos relacionados também foi concluído, permitindo identificar uma aparente falta de estudos que comparem empiricamente MVC e Hexagonal mantendo constantes o *runtime*, o *framework* HTTP e o banco de dados, e variando apenas a arquitetura. Essa constatação reforça a pertinência da pesquisa proposta.

O objetivo relativo ao desenvolvimento de uma aplicação equivalente nas duas arquiteturas foi parcialmente atendido. Nesta etapa, consolidaram-se a justificativa do domínio, o modelo de dados, os casos de uso, as regras de negócio, os *endpoints* e a estrutura interna de cada implementação, conforme descrito no Capítulo 4. A codificação completa das duas APIs, contudo, será concluída no TCC 2. Os objetivos referentes à mensuração do desempenho, à avaliação da manutenibilidade e à análise comparativa dos resultados dependem da execução dos experimentos e, portanto, serão cumpridos na etapa subsequente.

Como contribuição parcial, este trabalho oferece um desenho experimental controlado e reproduzível, no qual a manutenibilidade é avaliada por meio de métricas objetivas extraídas automaticamente do código-fonte análise de acoplamento, testabilidade e custo de substituição de componentes, reduzindo a subjetividade frequentemente associada a esse tipo de avaliação. Espera-se que os resultados forneçam subsídios concretos para a tomada de decisão arquitetural em projetos baseados em Node.js, Express e PostgreSQL.

Como encaminhamento para o TCC 2, prevê-se a implementação completa das duas aplicações com paridade funcional, a execução dos cenários de *benchmarking* nos diferentes níveis de carga, a coleta das métricas de manutenibilidade e a análise dos resultados, de modo a responder integralmente à questão de pesquisa. Como evolução futura, considera-se ainda a incorporação de métodos estatísticos mais robustos à análise dos dados coletados. O planejamento detalhado dessas atividades é apresentado no cronograma a seguir.

As atividades previstas para a conclusão da pesquisa no TCC 2 estão distribuídas ao longo do segundo semestre de 2026, conforme apresentado no Tabela 15. O planejamento con-

templa a implementação completa das duas aplicações, a execução dos experimentos de desempenho e de manutenibilidade, a análise dos resultados e a redação final do trabalho.

Tabela 15 – Cronograma de atividades previstas para o TCC 2 (2026/2)

Atividade	Ago	Set	Out	Nov	Dez
Implementação da API MVC	x	x			
Implementação da API Hexagonal	x	x			
Configuração do ambiente e do <i>autocannon</i>		x			
Execução dos cenários de teste (leitura, escrita, mista)		x	x		
Coleta e análise das métricas de desempenho			x		
Avaliação de manutenibilidade			x	x	
Análise e discussão dos resultados				x	
Redação dos capítulos de Resultados e Conclusão				x	x
Revisão geral e ajustes finais					x

Fonte: O Autor (2026).

REFERÊNCIAS

- BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. Addison-Wesley, 2012. Acessado em: 25 mar. 2026. Disponível em: <<https://ptgmedia.pearsoncmg.com/images/9780321815736/samplepages/0321815734.pdf>>.
- BELLACOSA, V. **Spaghetti Code: um código sem nexo, o emaranhado de comandos que acabará com sua sanidade**. 2021. Disponível em: <<https://www.dio.me/articles/spaghetti-code-um-codigo-sem-nexo-emaranhado-de-comandos>>. Acesso em: 16 mar. 2026.
- CHACON, S.; STRAUB, B. **Pro Git**. 2. ed. Apress, 2014. Disponível em: <<https://git-scm.com/book/en/v2>>. Acesso em: 17 jun. 2026.
- COCKBURN, A. **Hexagonal Architecture**. 2005. Disponível em: <<https://alistair.cockburn.us/hexagonal-architecture/>>. Acesso em: 12 mar. 2026.
- DAMIR, S.; SERGII, M. **Ready for changes with Hexagonal Architecture**. 2020. Disponível em: <<https://netflixtechblog.com/ready-for-changes-with-hexagonal-architecture-b315ec967749>>. Acesso em: 17 mar. 2026.
- DIJKSTRA, E. W. **On the Role of Scientific Thought**. [S.l.]: Springer, 1974. Acessado em: 26 mar. 2026.
- EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2003. Acessado em: 08 abr. 2026.
- FASTIFY. **Benchmarks**. 2024. Acessado em: 09 abr. 2026. Disponível em: <<https://fastify.dev/benchmarks>>.
- HOMER, P. W. **Spaghetti Code**. 2009. Disponível em: <<https://theprogrammersparadox.blogspot.com/2009/11/spaghetti-code.html>>. Acesso em: 16 mar. 2026.
- INFOQ. **Facebook: MVC Does Not Scale, Use Flux Instead**. 2014. Disponível em: <<https://www.infoq.com/news/2014/05/facebook-mvc-flux/>>. Acesso em: 18 mar. 2026.
- INTEGRAL.IO. **Hexagonal Architecture Transformation**. 2024. Disponível em: <<https://integral.io/insights/blog/hexagonal-architecture-transformation/>>. Acesso em: 18 mar. 2026.
- ISO/IEC. **ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models**. 2011. Disponível em: <<https://www.iso.org/standard/35733.html>>. Acesso em: 16 abr. 2026.
- JAIN, R. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling**. [S.l.]: Wiley, 1991. (Wiley Professional Computing). Acessado em: 29 abr. 2026.
- LEACH, P.; MEALLING, M.; SALZ, R. **A Universally Unique Identifier (UUID) URN Namespace**. [S.l.], 2005. Disponível em: <<https://www.rfc-editor.org/rfc/rfc4122>>. Acesso em: 17 jun. 2026.

LEMOS, M. F. de *et al.* Aplicabilidade da arquitetura MVC em uma aplicação web (WebApps). **Revista RE3C**, 2013. Disponível em: <<https://revistas.unifenas.br/index.php/RE3C/article/view/54>>. Acesso em: 28 jun. 2026.

LIMA, B. S. C. **Análise Comparativa de Fatores de Desempenho na Adoção de Frameworks de Desenvolvimento Web Backend**. Belo Horizonte: [s.n.], 2023. Acessado em: 09 abr. 2026. Disponível em: <<http://bib.pucminas.br:8080/pergamumweb/vinculos/0000bb/0000bb01.pdf>>.

MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Upper Saddle River: Prentice Hall, 2017. Acessado em: 26 mar. 2026. Disponível em: <<https://github.com/GunterMueller/Books-3/blob/master/Clean%20Architecture%20A%20Craftsman%20Guide%20to%20Software%20Structure%20and%20Design.pdf>>.

MIGUEL, R. **Arquitetura Hexagonal é overengineering?** 2025. Disponível em: <<https://chroniclesofapragmaticprogrammer.substack.com/p/arquitetura-hexagonal-e-overengineering>>. Acesso em: 16 mar. 2026.

MILET, P. de M. **Benchmark de runtimes Javascript**. Dissertação (Mestrado) — Universidade Federal de Pernambuco, Recife, 2024. Acessado em: 09 abr. 2026. Disponível em: <<https://repositorio.ufpe.br/handle/123456789/57491>>.

PALERMO, J. **The Onion Architecture: Part 1**. 2008. Blog post. Acessado em: 26 mar. 2026. Disponível em: <<https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/>>.

PEREIRA, L. G. N.; DINIZ, L. M. F. **Comparativo entre Arquitetura Monolítica e Arquitetura de Microsserviços**. Betim: [s.n.], 2022. Acessado em: 09 abr. 2026. Disponível em: <<http://bib.pucminas.br:8080/pergamumweb/vinculos/000018/00001811.pdf>>.

REENSKAUG, T. **The Model-View-Controller (MVC): Its Past and Present**. 1979. Disponível em: <<http://heim.ifi.uio.no/~trygver/themes/mvc/mvc-index.html>>. Acesso em: 12 mar. 2026.

SANTOS, A. L. G. dos *et al.* **Análise de Desempenho de Frameworks para Aplicações Distribuídas**. Itajaí: [s.n.], 2025. 487–493 p. Acessado em: 09 abr. 2026. Disponível em: <<https://periodicos.univali.br/index.php/acotb/article/view/21091>>.

SOMMERVILLE, I. **Engenharia de Software**. São Paulo: Pearson Prentice Hall, 2011. Acessado em: 25 mar. 2026. Disponível em: <<https://www.facom.ufu.br/~william/Disciplinas%202018-2/BSI-GSI030-EngenhariaSoftware/Livro/engenhariaSoftwareSommerville.pdf>>.

SVERWEIJ, S. **dependency-cruiser**. 2024. Disponível em: <<https://github.com/sverweij/dependency-cruiser>>. Acesso em: 16 abr. 2026.

TEAM, J. C. **Jest API Reference**. 2025. Acessado em: 16 abr. 2026. Disponível em: <<https://jestjs.io/docs/api>>.

UCHITELLE, E.; THOMAS, C. **Building GitHub with Ruby and Rails**. 2023. Disponível em: <<https://github.blog/engineering/architecture-optimization/building-github-with-ruby-and-rails/>>. Acesso em: 17 mar. 2026.

APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Em conformidade com as Diretrizes para o uso ético, responsável e seguro de Inteligência Artificial no Ensino Superior (UCS, 2026), declaro que foram utilizadas ferramentas de Inteligência Artificial generativa como ferramenta de apoio durante a elaboração deste trabalho acadêmico.

As ferramentas utilizadas incluíram o ChatGPT (OpenAI, versão GPT-5) e Claude (Anthropic, versão Claude Sonnet 4), empregados exclusivamente para apoio na reescrita, reorganização e refinamento textual de conteúdos previamente elaborados manualmente pelo autor.

Declara-se portanto que a utilização ocorreu apenas como instrumento de apoio à escrita acadêmica, não substituindo a autoria intelectual da pesquisa, envolvendo:

- revisão gramatical e ortográfica;
- melhoria de clareza e fluidez textual;
- adequação da escrita ao padrão acadêmico;
- reorganização estrutural de trechos redigidos manualmente.